المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵉⵎⵙⵙⵓⵔⵉⵏ ⵜⵓⵏⴰⵙⵜⵉⵜ ⵏ ⵜⵙⵏⴰⵡⵉⵏ ⵜⵉⵜⴰⵡⵉⵏⵉⵏ ⴰⵜⵜⴰⵡⵉⵏ
Ecole Nationale des Sciences Appliquées de Tétouan

Projet de Fin d'Année

Filière : Génie Informatique

Année : 2025 – 2026

Sujet :

Alternative Open-Source à TKI PGP Migration d'un Industry Model Autodesk vers PostgreSQL/PostGIS

---

oec projekt GmbH

InfraNet ENGINEERING

Réalisé par : Ameziane Oumaima – Mohito Raihana

Encadré par :

- Encadrant Pédagogique : Mr. Abderrahim TAHIRI
- Encadrants Professionnels : - Mme. Constanze Weißleder
  - Mr. Christian Heyne

Résumé

Ce projet de fin d'année, réalisé dans le cadre d'un stage professionnel au sein de **Infranet Engineering SARL** , filiale marocaine de la société allemande **CEC Projekt GmbH** et nouvelle entité de l'**Infra Group**, porte sur l'étude, la conception et la validation d'une alternative open-source au connecteur commercial **TKI PGP**, utilisé pour exploiter des *Industry Models* Autodesk sur une base de données **PostgreSQL/PostGIS**. L'objectif principal consiste à évaluer la faisabilité technique d'une solution équivalente reposant exclusivement sur des composants libres, tout en préservant les principales fonctionnalités nécessaires à l'exploitation et à l'édition des données géospatiales, sans dépendre d'un logiciel propriétaire sous licence.

La démarche adoptée s'est articulée autour de six phases complémentaires. Elle a débuté par l'étude de l'architecture Autodesk existante, notamment *Infrastructure Administrator* et **AutoCAD Map 3D**, suivie de la validation d'une connexion directe entre Map 3D et PostgreSQL/PostGIS à travers le connecteur **FDO** natif. Une phase de **rétro-ingénierie** différentielle du *Data Model* Autodesk, stocké dans une base SQLite, a ensuite permis d'identifier et d'analyser les structures nécessaires à sa représentation. Cette analyse a été complétée par une étude fonctionnelle du rôle de **TKI PGP**, afin de distinguer les fonctionnalités indispensables des mécanismes spécifiques au connecteur commercial. Sur cette base, une architecture cible open-source a été définie, puis implémentée et validée au moyen de tests de bout en bout.

La solution développée repose principalement sur deux composants complémentaires : un générateur de schéma DDL développé en **Python**, s'appuyant sur les six catalogues maîtres identifiés lors de la rétro-ingénierie, et le connecteur FDO natif d'AutoCAD Map 3D, permettant l'accès et l'édition des données géospatiales directement dans PostgreSQL/PostGIS. Un prototype fonctionnel a été développé et validé sur un *Data Model* de test. Les essais réalisés ont permis de générer automatiquement un schéma PostgreSQL/PostGIS complet et cohérent, puis de vérifier son exploitation à travers l'environnement Autodesk.

Les résultats obtenus confirment ainsi la faisabilité technique d'une alternative open-source à TKI PGP. Au-delà de la validation du concept, ce travail fournit une base technique pouvant être enrichie et industrialisée, notamment par l'automatisation de la migration complète des modèles Autodesk, l'amélioration de la couverture fonctionnelle et l'intégration de mécanismes supplémentaires nécessaires à une utilisation en environnement de production.

Mots-clés : Industry Model Autodesk, PostgreSQL, PostGIS, AutoCAD Map 3D, FDO, rétro-ingénierie, open-source, TKI PGP.

Abstract

This end-of-year project, carried out as part of a professional internship at **Infranet Engineering SARL** , a Moroccan subsidiary of the German company **CEC Projekt GmbH** and a new entity of the **Infra Group**, focuses on the study, design, and validation of an open-source alternative to the commercial **TKI PGP** connector, which is used to operate Autodesk *Industry Models* on a **PostgreSQL/PostGIS** database. The main objective is to assess the technical feasibility of an equivalent solution based exclusively on open-source components, while preserving the key functionalities required for the management and editing of geospatial data, without relying on proprietary licensed software.

The adopted approach was structured into six complementary phases. It began with an analysis of the existing Autodesk architecture, particularly *Infrastructure Administrator* and **AutoCAD Map 3D**, followed by the validation of a direct connection between Map 3D and PostgreSQL/PostGIS through the native **FDO** connector. A phase of controlled differential **reverse engineering** of the Autodesk *Data Model*, stored in a SQLite database, was then conducted to identify and analyze the structures required for its representation. This analysis was complemented by a functional study of the role of **TKI PGP**, in order to distinguish essential functionalities from mechanisms specific to the commercial connector. Based on these findings, a target open-source architecture was designed, implemented, and validated through end-to-end testing.

The developed solution relies mainly on two complementary components : a Python-based DDL schema generator built upon the six master catalogs identified during the reverse-engineering process, and the native FDO connector of AutoCAD Map 3D, which enables direct access to and editing of geospatial data in PostgreSQL/ PostGIS. A functional prototype was developed and successfully validated using a test *Data Model*. The conducted tests demonstrated the automatic generation of a complete and consistent PostgreSQL/PostGIS schema, followed by its successful use within the Autodesk environment.

The results obtained confirm the technical feasibility of an open-source alternative to TKI PGP. Beyond validating the concept, this work provides a technical foundation that can be further enhanced and industrialized, particularly through the automation of complete Autodesk Data Model migration, the extension of functional coverage, and the integration of additional mechanisms required for production environments.

Keywords : Autodesk Industry Model, PostgreSQL, PostGIS, AutoCAD Map 3D, FDO, reverse engineering, open-source, TKI PGP.

Table des figures

1 Création d'un Industry Model de test . . . 14
2 Ouverture de la boîte de dialogue de connexion dans AutoCAD Map 3D . 15
3 Sélection de l'industry model à connecter, . . . 16
4 Résultat de la connexion . . . 16
5 Architecture interne entre Autocad et Infrastructure Administrator observée . . . 17
6 Base de données de test : map3d_test . . . 18
7 Fenêtre de connexion PostgreSQL — saisie des identifiants pour la base map3d_test . . . 19
8 Panneau « Add Data to Map » — sélection des couches du schéma fibre à ajouter au dessin . . . 20
9 Sélection de l'objet 'CH-002' dans le dessin — correspondance confirmée avec la base . . . 20
10 Vérification en base via pgAdmin — les attributs modifiés dans Map 3D sont bien persistés dans fibre.chambre . . . 20
11 Diagramme d'activité illustrant le fonctionnement séquentiel du script de comparaison des tests . . . 26
12 Arborescence complète du Data Model dans Infrastructure Administrator . 27
13 Liste des attributs d'une classe (exemple de la classe `Construct Point`) . . 28
14 État initial du fichier SQLite dans DB Browser for SQLite – 170 tables présentes par défaut . . . 28
15 Site Officiel du TKI Engineering . . . 33
16 Architecture cible de la solution : du Data Model Autodesk à AutoCAD Map 3D . . . 37
17 Diagramme d'activité illustrant le fonctionnement séquentiel du moteur de conversion DDL . . . 40
18 Diagramme d'activité illustrant la boucle de surveillance du service `watch_and_sync.py` 43
19 Diagramme d'activité illustrant le traitement d'un modèle Autodesk détecté 44
20 Résultats des tests unitaires . . . 45
21 Exécution et validation du pipeline de synchronisation en ligne de commande (CLI) . . . 46
22 Interface de l'application graphique d'administration System Tray . . . 47
23 État Actif (Vert) . . . 48
24 État En pause (Orange) . . . 48
25 Menu contextuel interactif . . . 48
26 Diagramme d'activité illustrant la chaîne de compilation du script `build_exe.py` en exécutable Windows autonome via PyInstaller . . . 49
27 Assistant d'installation — Étape d'accueil de Setup_PostMapLive.exe . . . 52
28 Assistant d'installation — Choix du répertoire de destination . . . 53
29 Assistant d'installation — Sélection du dossier du menu Démarrer . . . 53
30 Assistant d'installation — Sélection des tâches complémentaires . . . 54
31 Assistant d'installation — Récapitulatif avant installation . . . 55
32 Assistant d'installation — Finalisation et lancement du service . . . 55
33 Localisation de l'icône System Tray de PostMap Live dans la barre des tâches Windows . . . 56
34 Menu contextuel — Service actif, option de mise en pause (*Pause Sync*) . . 57

35 Menu contextuel — Service en pause, option de reprise (*Resume Sync*) . . 57
36 Accès à la fenêtre de configuration via le menu contextuel (*Settings*) 58
37 Fenêtre de configuration PostMap Live — Paramètres de connexion PostgreSQL et code SRID 58
38 Accès à la fenêtre des journaux via le menu contextuel (*View Logs*) 59
39 Fenêtre des journaux en temps réel — Détection du modèle, génération et application du schéma PostGIS 59
40 Classes métier synchronisées affichées dans AutoCAD Map 3D (`Drawing_IM_initial_1`) 60
41 Classes métier synchronisées affichées dans AutoCAD Map 3D (`Drawing_IM_initial_2`) 61
42 Vérification des données synchronisées sous pgAdmin — table `TEST_CLASSE_GEO_08` avec géométrie et longueur calculée automatiquement 61
43 Capture d'écran des services lancés 64
44 Capture d'ecran de l'interface web Grafana (http ://localhost :3000) . . . 65
45 Capture d'écran de Jira 66
46 Diagramme de Gantt du projet (juillet – août 2026) 67
47 Capture d'écran de Slack 68
48 Capture d'écran du dépôt Github 68

Liste des tableaux

1 Fonctions principales de `compare_sqlite.py` 24
2 Les six catalogues maîtres du Data Model Autodesk 30
3 Catégorisation des tables SQLite observées 31
4 Évaluation comparative des approches techniques envisagées 36
5 Mapping des six catalogues maîtres vers PostgreSQL 38
6 Suite de tests unitaires et d'intégration automatisés 45
7 Sections principales du script Inno Setup `installer_setup.iss` 50
8 Arborescence et rôle des fichiers installés sur le poste client 62
9 Architecture des conteneurs de la stack d'observabilité 63

Table des matières

Introduction générale **9**

1 Contexte général et problématique **11**
1.1 Contexte . . . 11
1.2 Problématique . . . 11
1.3 Objectifs du projet . . . 11
1.4 Périmètre et limites . . . 11
1.5 Méthodologie générale . . . 12

2 Découverte de l'architecture Autodesk **13**
2.1 Infrastructure Administrator . . . 13
2.2 Création d'une Fachschale de test . . . 13
2.3 AutoCAD Map 3D côté client . . . 14
2.4 Synthèse . . . 16

3 Environnement PostgreSQL/PostGIS et connecteur FDO natif **18**
3.1 Mise en place de l'environnement . . . 18
3.2 Schéma de démonstration . . . 19
3.3 Test du connecteur FDO natif . . . 19
3.4 Ce que le connecteur FDO apporte – et n'apporte pas . . . 21
3.5 Conclusion de la phase . . . 21

4 Rétro-ingénierie du Data Model Autodesk **22**
4.1 Objectif et enjeux de la phase . . . 22
4.2 Méthodologie . . . 22
4.3 Fonctionnement du script de comparaison automatisée . . . 23
4.3.1 Les grandes étapes du traitement . . . 23
4.3.2 Un point technique déterminant : la gestion des blocs de trigger . . . 24
4.3.3 Sortie produite . . . 25
4.4 Campagne de tests . . . 27
4.4.1 État de référence (Test 0) . . . 27
4.4.2 Création de classes (Tests 1, 7 et 8) . . . 28
4.4.3 Attributs et contraintes (Tests 2 à 6) . . . 29
4.4.4 Relations et domaines de valeurs (Tests 9 à 11) . . . 29
4.4.5 Héritage entre classes (Test 12) . . . 29
4.4.6 Renommage et suppression (Tests 16 à 18) . . . 30
4.5 Résultat : les six catalogues maîtres . . . 30
4.6 Catégorisation des tables SQLite . . . 30
4.7 Périmètre exclu de l'analyse . . . 31
4.8 Conclusion . . . 32

5 Rôle de TKI PGP dans l'architecture Autodesk **33**
5.1 Présentation . . . 33
5.2 Analyse fonctionnelle . . . 33
5.3 Questions ouvertes . . . 34
5.4 Synthèse et pistes pour la solution alternative . . . 34

6 Architecture cible et conception de la solution alternative 35
6.1 Objectif et exigences fonctionnelles . . . 35
6.2 Étude comparative et évaluation des approches . . . 35
6.3 Architecture hybride retenue (Combinaison A + E) . . . 36
6.4 Conclusion du chapitre . . . 37

7 Implémentation, synchronisation et validation du moteur 38
7.1 Moteur de conversion DDL . . . 38
7.1.1 Mapping des 6 catalogues maîtres vers PostgreSQL . . . 38
7.1.2 Résolution algorithmique des règles complexes . . . 39
7.1.3 Fonctionnement séquentiel du moteur de conversion DDL . . . 39
7.2 Moteur de surveillance et synchronisation temps réel . . . 41
7.2.1 Algorithme de détection dynamique dans `%TEMP%` . . . 41
7.2.2 Algorithme de validation à 5 niveaux (`is_autodesk_sqlite`) . . . 41
7.2.3 Synchronisation non destructive . . . 41
7.2.4 Fonctionnement séquentiel du moteur de surveillance et de synchronisation . . . 42
7.3 Validation par tests automatisés (Suite Pytest) . . . 45
7.4 Validation par tests manuels et commandes CLI . . . 45
7.4.1 Commande de lancement manuelle . . . 46
7.4.2 Scénarios de modification et de suppression physique . . . 46

8 Application d'administration, packaging et déploiement Windows 47
8.1 Interface d'administration System Tray (`gui_tray_app.py`) . . . 47
8.1.1 Gestion des états et icônes dynamiques . . . 47
8.1.2 Fonctionnalités du menu contextuel . . . 48
8.2 Procédure de packaging et compilation autonome (`build_exe.py`) . . . 48
8.2.1 Fonctionnement et chaîne de compilation du script `build_exe.py` . . . 48
8.2.2 Gestion de l'arborescence temporaire runtime . . . 49
8.3 Création de l'installateur Windows Inno Setup . . . 50
8.3.1 Structure et paramètres clés du script Inno Setup (`installer_setup.iss`) 50
8.3.2 Procédure de génération de l'installateur . . . 50
8.4 Synthèse du chapitre . . . 51

9 Guide d'installation et d'utilisation 52
9.1 Guide d'installation pas à pas (`Setup_PostMapLive.exe`) . . . 52
9.2 Guide d'utilisation au quotidien . . . 56
9.2.1 Localisation et accès à l'application . . . 56
9.2.2 Menu contextuel et suivi de l'état du service . . . 56
9.2.3 Configuration initiale des accès PostgreSQL . . . 57
9.2.4 Consultation des journaux de synchronisation . . . 58
9.2.5 Workflow de synchronisation automatique et édition sous AutoCAD Map 3D . . . 60
9.2.6 Indépendance des dessins et isolation multi-fichiers DWG . . . 61
9.3 Organisation des fichiers installés . . . 62
9.4 Synthèse du guide . . . 62

10 Conteneurisation et observabilité serveur 63
10.1 Motivation et objectifs d'observabilité . . . 63

10.2 Architecture conteneurisée unifiée (`docker-compose.yml`) 63
10.3 Mécanisme de collecte et configuration Prometheus (`prometheus.yml`) . . 64
10.4 Supervision visuelle et tableaux de bord Grafana 64
10.5 Journalisation, persistance et sauvegardes (`pg_dump`) 65
10.6 Synthèse et bilan d'infrastructure 65

11 Gestion du projet et collaboration **66**
11.1 Méthodologie de gestion de projet 66
11.2 Suivi des tâches 66
11.3 Planification du projet 67
11.4 Communication et collaboration 67
11.5 Gestion des versions et documentation 68
11.6 Synthèse 69

Conclusion générale et perspectives **70**

Références **71**

Introduction générale

Dans un contexte où la transformation numérique des infrastructures critiques s'accélère, la maîtrise des données géospatiales constitue un enjeu stratégique majeur pour les bureaux d'études et les gestionnaires de réseaux. La capacité à stocker, exploiter et mettre à jour ces données de manière efficace et souveraine conditionne directement la réactivité opérationnelle et la performance des projets d'aménagement. Dans cet écosystème, les *Industry Models* d'Autodesk se sont imposés comme des références pour la modélisation des réseaux (eau, gaz, électricité, télécommunications), mais leur dépendance aux bases de données propriétaires (Oracle, SQL Server) ou à des connecteurs commerciaux tiers comme TKI PGP introduit des contraintes de coût et de dépendance technologique qui freinent l'innovation.

Au sein de l'entreprise d'accueil, Infranet Engineering SARL , la problématique principale identifiée concerne l'exploitation d'un *Industry Model* Autodesk sur PostgreSQL/PostGIS. Si ce système de gestion de base de données open-source est plébiscité pour sa robustesse, sa scalabilité et son extension spatiale mature, il n'est pas officiellement supporté par Autodesk pour les modèles de données métier. Le recours au connecteur **TKI PGP**, bien que fonctionnel, représente un coût de licence récurrent et une dépendance à un éditeur externe, limitant ainsi la flexibilité et la souveraineté technique de l'entreprise. De plus, les mécanismes internes de traduction du *Data Model* Autodesk vers un schéma relationnel restent opaques, rendant toute évolution ou adaptation technique complexe.

Afin de répondre à ces besoins, le projet proposé consiste à étudier, concevoir et valider une alternative open-source à TKI PGP permettant de :

— Comprendre et documenter la structure interne du *Data Model* Autodesk via une rétro-ingénierie différentielle contrôlée.
— Développer un générateur de schéma DDL en Python capable de transformer automatiquement le modèle stocké en SQLite vers PostgreSQL/PostGIS.
— Exploiter le connecteur **FDO** (*Feature Data Objects*) natif d'AutoCAD Map 3D pour assurer l'édition bidirectionnelle et temps réel des données géospatiales.
— Mettre en place un service de surveillance et de synchronisation garantissant la cohérence entre les modifications du modèle et la base de données cible.
— Valider l'ensemble par des tests de bout en bout sur un modèle de test représentatif.

La solution technique retenue repose sur une architecture alliant légèreté et contrôle : **Python** pour le moteur de conversion (avec des bibliothèques standard pour l'introspection SQLite), **PostgreSQL/PostGIS** pour le stockage des données, et le connecteur FDO intégré à **AutoCAD Map 3D** pour la couche d'édition cartographique. La synchronisation est automatisée via un script de surveillance, et l'industrialisation est assurée par un packaging exécutable (**PyInstaller**), un installateur Windows (**Inno Setup**) et une interface graphique résidant en barre des tâches (**tkinter** et **pystray**). Des outils de versionnage (Git) et de test (**Pytest** avec bases de test dédiées) complètent l'environnement de développement.

Objectifs du projet :

L'objectif principal de ce travail est d'évaluer la faisabilité technique d'une solution alternative à TKI PGP. Plus spécifiquement, il s'agit de :

— Fournir une cartographie claire et documentée des catalogues maîtres constituant le *Data Model* Autodesk.
— Offrir un outil capable de générer un schéma PostgreSQL/PostGIS strictement équivalent à partir de n'importe quelle (*Industry Model*) Autodesk.
— Démontrer que l'édition cartographique et attributaire en temps réel est possible sans plugin tiers, via le connecteur FDO natif.
— Proposer une base technique industrialisable pour une future migration complète vers l'open-source, réduisant les coûts de licence et la dépendance à des éditeurs commerciaux.

Structure du rapport

Le présent rapport est structuré comme suit :

— **Chapitre 1 : Contexte général et problématique** – Présentation du cadre industriel, des enjeux d'indépendance technologique et des objectifs du projet.
— **Chapitre 2 : Découverte de l'architecture Autodesk** – Étude de l'écosystème Autodesk, d'Infrastructure Administrator, d'AutoCAD Map 3D et de la modélisation sous SQLite.
— **Chapitre 3 : Environnement PostgreSQL/PostGIS et connecteur FDO natif** – Validation de la connexion directe entre AutoCAD Map 3D et PostGIS via le provider FDO natif sans plugin tiers.
— **Chapitre 4 : Rétro-ingénierie du Data Model Autodesk** – Démarche du différentiel contrôlé, campagne de tests et identification des six catalogues maîtres.
— **Chapitre 5 : Rôle de TKI PGP dans l'architecture Autodesk** – Analyse fonctionnelle du connecteur commercial et identification des exigences clés à reproduire.
— **Chapitre 6 : Architecture cible et conception de la solution alternative** – Spécification de l'approche hybride retenue (A + E) et algorithmes du moteur DDL Python.
— **Chapitre 7 : Implémentation, synchronisation et validation du moteur** – Moteur de conversion DDL, moteur de surveillance temps réel, et validation par tests automatisés et manuels.
— **Chapitre 8 : Application d'administration, packaging et déploiement Windows** – Interface System Tray, compilation autonome via PyInstaller et installateur Windows Inno Setup.
— **Chapitre 9 : Guide d'installation et d'utilisation** – Installation pas à pas, configuration PostgreSQL et utilisation quotidienne sous AutoCAD Map 3D.
— **Chapitre 10 : Conteneurisation et observabilité serveur** – Infrastructure Docker (PostgreSQL/PostGIS, Prometheus, Grafana) et supervision des métriques.
— **Chapitre 11 : Gestion du projet et collaboration** – Méthodologie de gestion, suivi des tâches, communication d'équipe et gestion des versions.

Enfin, une conclusion récapitulera les apports du projet et proposera des perspectives d'industrialisation et d'extension fonctionnelle.

Ainsi, ce projet s'inscrit dans une démarche d'innovation technique visant à doter l'entreprise d'une alternative crédible et performante, tout en renforçant son autonomie vis-à-vis des solutions propriétaires dans le domaine stratégique de la gestion des données d'infrastructure.

Chapitre 1: Contexte général et problématique

Ce chapitre pose le cadre général du projet en présentant le contexte industriel, la problématique spécifique liée à l'absence de support natif de PostgreSQL pour les Industry Models Autodesk, ainsi que les objectifs, le périmètre et la méthodologie adoptée.

1.1 Contexte

AutoCAD Map 3D, couplé à Autodesk Infrastructure Administrator, permet de créer des **Industry Models**, des modèles de données métier adédiés à la gestion de réseaux d'infrastructure (eau, électricité, gaz, télécommunications, etc.). Officiellement, un Industry Model basé sur base de données ne peut être créé que sur Oracle ou Microsoft SQL Server. PostgreSQL, bien qu'il s'agisse d'un système de gestion de bases de données relationnelles open source robuste, doté avec PostGIS d'une extension spatiale mature, n'est pas proposé nativement dans ce cadre.

Pour combler cet écart, la société TKI commercialise un connecteur, **TKI PGP**, qui permet d'utiliser PostgreSQL/PostGIS comme moteur de stockage d'un Industry Model. Ce connecteur reste toutefois un produit commercial, sous licence, avec les contraintes de coût et de dépendance à un éditeur tiers que cela implique.

1.2 Problématique

La problématique centrale du projet peut être formulée ainsi : comment permettre le stockage et l'exploitation d'un Industry Model Autodesk sous PostgreSQL/PostGIS, avec un niveau fonctionnel comparable à celui de TKI PGP (création du schéma, lecture, création, mise à jour et suppression d'objets depuis AutoCAD Map 3D), sans recourir à un produit commercial ?

1.3 Objectifs du projet

Cinq objectifs ont guidé l'ensemble de la démarche :

1. Comprendre en détail le fonctionnement interne du **Data Model** Autodesk, c'est-à-dire la manière dont les classes d'objets, attributs, géométries, domaines et relations sont traduits en structures relationnelles.
2. Comprendre le périmètre fonctionnel exact de **TKI PGP**, à partir de sa documentation publique et de son comportement observable, sans rétro-ingénierie de son code.
3. Concevoir et comparer plusieurs architectures possibles pour une solution alternative.
4. Développer un outil capable de générer automatiquement un schéma **PostgreSQL/-PostGIS** équivalent à partir d'un Data Model Autodesk.
5. Valider la solution à travers un scénario de bout en bout dans AutoCAD Map 3D.

1.4 Périmètre et limites

Le projet se concentre exclusivement sur **TKI PGP** ; la solution métier complète **TKI NET** sort du cadre du stage. Aucune rétro-ingénierie du code de TKI PGP n'a été réalisée : l'analyse repose uniquement sur la documentation publique et le comportement

observable du produit. Le développement s'appuie sur des Data Models de test créés spécifiquement pour le projet, et non sur des données de production.

1.5 Méthodologie générale

La compréhension du Data Model Autodesk repose sur une méthode de **rétro-ingénierie différentielle contrôlée** : chaque test consiste à effectuer une seule modification dans Infrastructure Administrator (ajout d'une classe, d'un attribut, d'une relation, etc.), puis à comparer automatiquement l'état du schéma SQLite avant et après, afin de déduire, par observation reproductible, la logique de correspondance entre le modèle conceptuel et sa représentation physique. Cette méthode, détaillée au Chapitre 4, constitue le socle méthodologique de l'ensemble du projet.

Chapitre 2: Découverte de l'architecture Autodesk

Ce chapitre décrit les deux outils principaux de l'écosystème Autodesk utilisés dans le projet : Infrastructure Administrator, qui permet de configurer les modèles de données, et AutoCAD Map 3D, le client de visualisation et d'édition. Il détaille également la création d'une Fachschale de test et la manière dont le modèle est traduit physiquement en base de données SQLite.

Cette phase visait à comprendre les deux outils Autodesk au cœur du projet : **Infrastructure Administrator**, qui gère la structure des données, et **AutoCAD Map 3D**, l'outil client utilisé pour visualiser et manipuler les données géospatiales.

2.1 Infrastructure Administrator

Infrastructure Administrator est un module intégré à **AutoCAD Map 3D** qui permet de configurer et de gérer les *Industry Models*, . Ces modèles permettent d'organiser et de gérer les données liées à différents domaines, comme les réseaux d'eau, de gaz, d'électricité ou d'assainissement. Infrastructure Administrator est un outil général qui permet de personnaliser ces modèles selon les besoins du projet, qu'ils utilisent des fichiers ou une base de données d'entreprise.

Grâce à ce module, l'utilisateur peut notamment créer et modifier la structure des données, définir les informations que chaque objet peut contenir, créer des formulaires de saisie, définir des règles de gestion et gérer les utilisateurs et leurs groupes. Il est donc utilisé pour adapter le fonctionnement de l'Industry Model aux besoins spécifiques de l'organisation.

Infrastructure Administrator est principalement organisé autour de deux outils complémentaires. Le premier est le **Data Model Administrator**, qui permet de définir la structure du modèle de données. Il permet notamment de créer ou modifier les classes d'objets, leurs attributs, les domaines de valeurs, les labels ainsi que les topologies. Par exemple, il peut servir à définir les informations associées à un objet représentant une conduite ou un équipement.

Le second est le **Form Designer**, qui permet de personnaliser les formulaires utilisés pour consulter ou saisir les données. Il permet de choisir les champs à afficher, leur organisation et leur présentation afin de rendre la saisie des informations plus simple pour l'utilisateur.

Cette organisation a été vérifiée directement dans l'interface d'**Infrastructure Administrator**. Toutefois, dans le cadre de notre étude, nous nous sommes principalement intéressés au **Data Model Administrator**, car il constitue l'outil central pour comprendre, créer et modifier la structure du modèle de données d'un industry model. Une attention particulière a ainsi été portée à la définition des classes d'objets, de leurs attributs et des différentes relations qui composent le modèle de données.

2.2 Création d'une Fachschale de test

Un *Industry Model* est un modèle de données métier créé via Infrastructure Administrator, qui définit l'ensemble des classes d'objets, de leurs attributs et de leurs règles pour un domaine d'infrastructure donné. Elle peut être basée sur base de données (Oracle ou SQL Server) ou basée sur fichier, auquel cas elle utilise SQLite comme moteur de stockage,

intégré dans un fichier DWG.

Une Fachschale de test a été créée pas à pas afin d'observer concrètement ce mécanisme. L'arborescence générée dans le Data Model Administrator fait apparaître les classes d'objets définies (*Construct Parent*, *Construct Line*, *Construct Marker*, *Construct Point*), organisées sous *Data model* > *Construct*, aux côtés des *Templates*, des *Domains* et des autres composants disponibles (topologies, intersections, système de référencement linéaire). Cette arborescence correspond à la définition logique du schéma.

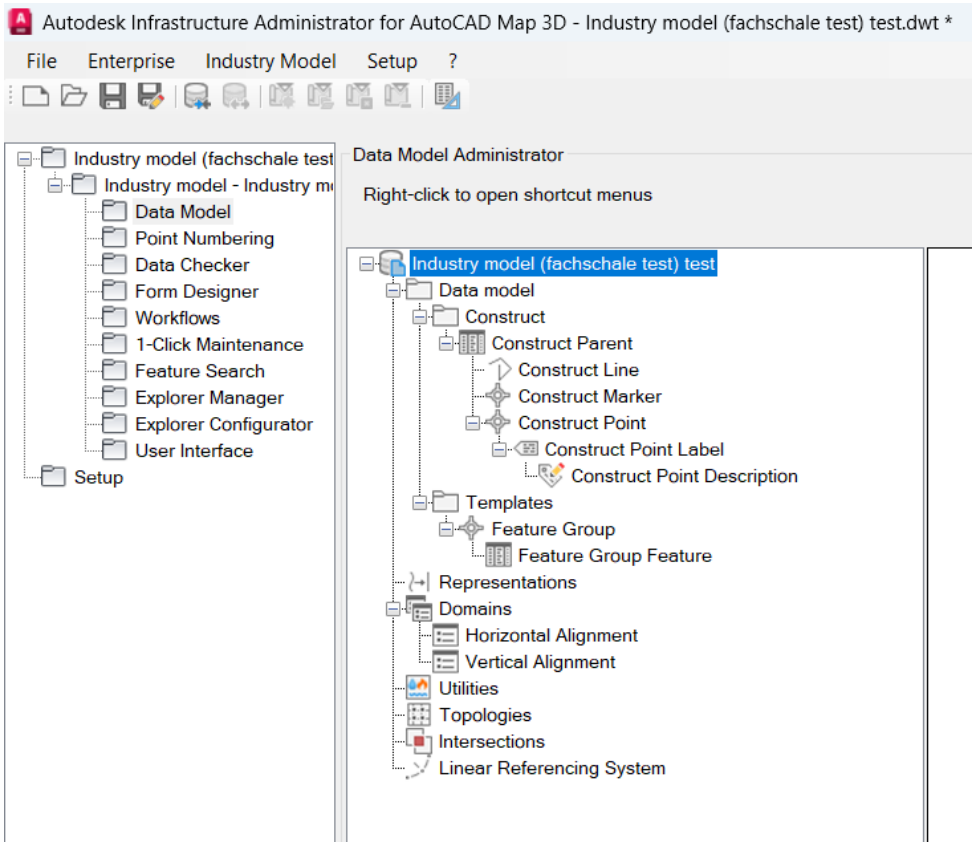


FIGURE 1 – Création d'un Industry Model de test

2.3 AutoCAD Map 3D côté client

AutoCAD Map 3D est un logiciel de cartographie et d'analyse SIG intégré à AutoCAD, combinant des fonctionnalités de DAO/CAO avec des outils SIG avancés. Il permet d'accéder directement aux données spatiales via la technologie **FDO** (*Feature Data Objects*), de modifier des données géospatiales et de gérer des modèles métier d'entreprise. C'est l'outil client qui exploite concrètement l'industry model créée dans Infrastructure Administrator : affichage de la carte, consultation et modification des attributs, connexion à un Industry Model existant.

La version installée sur le poste de travail (AutoCAD Map 3D 2025.0.2, construite sur AutoCAD 2025.1.3) correspond exactement à celle exigée par le cahier des charges du stage, confirmant la conformité de l'environnement de travail dès cette première phase.

La connexion à la Fachschale (Industry model) de test a été établie via la boîte de dialogue *Data Connect*, en sélectionnant le fournisseur permettant de se connecter à l'Industry

Model, puis en paramétrant le chemin du fichier correspondant. Une fois connectée, la Fachschale apparaît dans le *Display Manager* d'AutoCAD Map 3D, confirmant que le modèle de données créé dans Infrastructure Administrator est bien reconnu côté client. La sélection d'un objet sur la carte et l'affichage de son tableau d'attributs ont permis de confirmer le lien direct entre la structure définie dans la Fachschale et les données manipulées côté client.

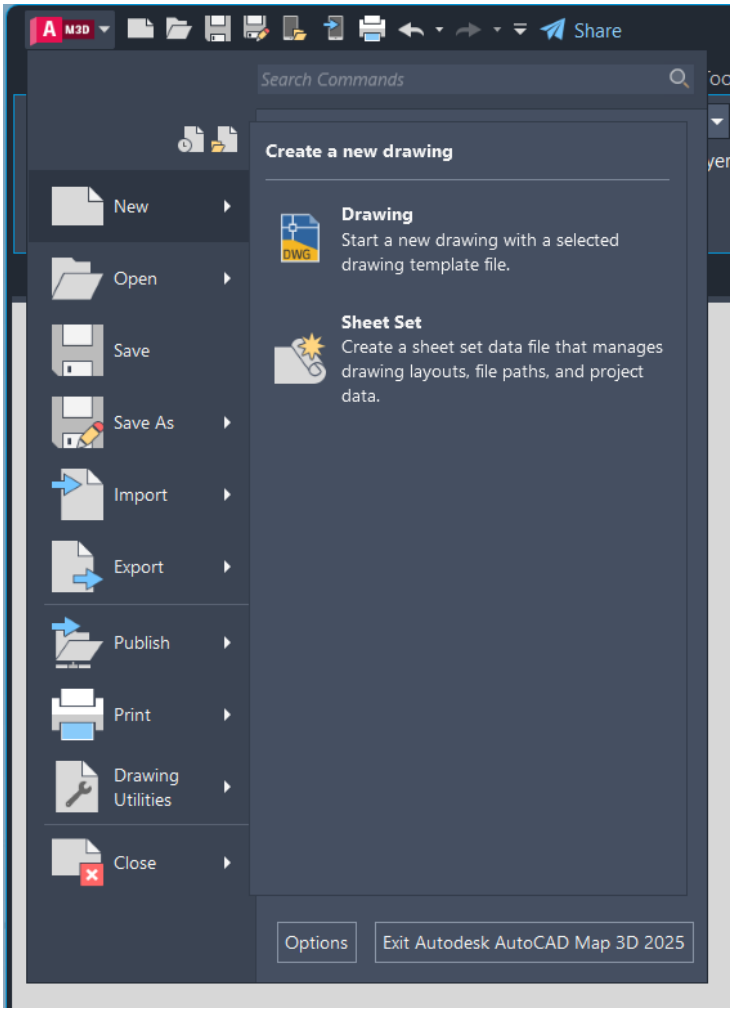


FIGURE 2 – Ouverture de la boîte de dialogue de connexion dans AutoCAD Map 3D

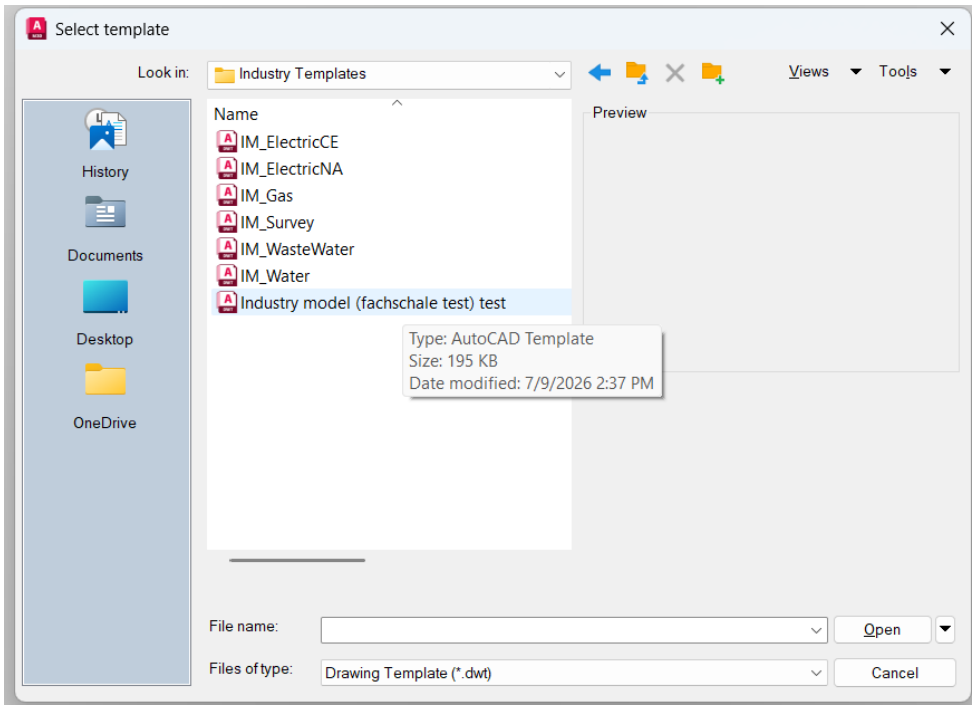


FIGURE 3 – Sélection de l'industry model à connecter,

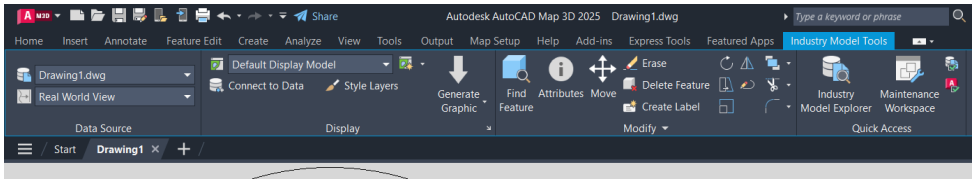


FIGURE 4 – Résultat de la connexion

L'industry model apparaît dans le Display Manager d'AutoCAD Map 3D, confirmant que le modèle de données créé dans Infrastructure Administrator est bien reconnu côté client.

2.4 Synthèse

L'Industry Model (Fachschale) créée dans Infrastructure Administrator définit le schéma métier du projet, matérialisé physiquement par un fichier SQLite en mode fichier. Ce même schéma devient directement exploitable côté client une fois la connexion établie dans AutoCAD Map 3D : les classes d'objets définies dans le Data Model Administrator sont celles que l'on retrouve et manipule concrètement sur la carte et dans les panneaux d'attributs. Cette compréhension de l'architecture officielle Autodesk constitue le point de départ nécessaire avant d'aborder la question de la migration vers PostgreSQL/PostGIS.

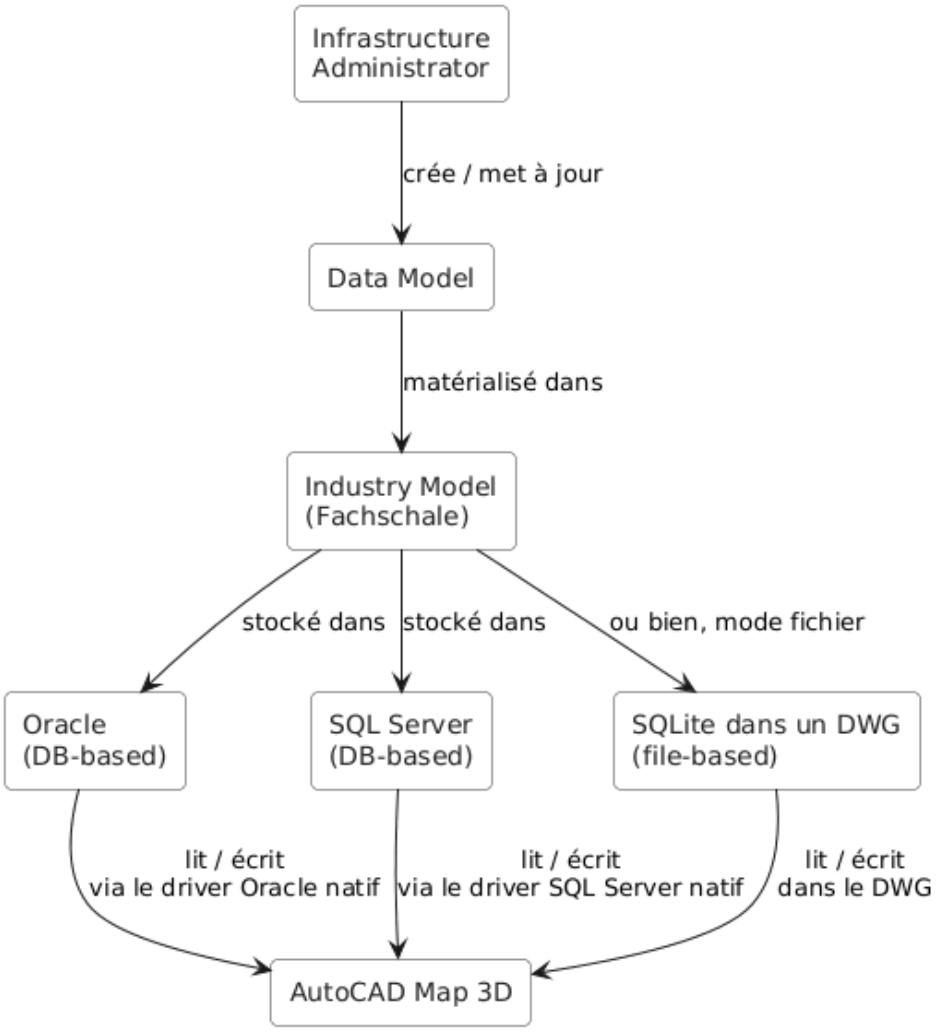


FIGURE 5 – Architecture interne entre Autocad et Infrastructure Administrator observée

Chapitre 3: Environnement PostgreSQL/PostGIS et connecteur FDO natif

Ce chapitre présente la mise en place d'une base de données PostgreSQL/PostGIS et valide la connexion directe entre AutoCAD Map 3D et cette base via le connecteur FDO natif. Il expose ce que ce connecteur apporte et ce qu'il ne couvre pas, positionnant ainsi les limites à dépasser.

Cette phase visait à mettre en place un environnement PostgreSQL/PostGIS et à déterminer si AutoCAD Map 3D pouvait communiquer directement avec cette base, sans passer par le connecteur commercial TKI PGP.

3.1 Mise en place de l'environnement

Une base de test, `map3d_test`, a été créée sous PostgreSQL, avec l'extension PostGIS installée (`CREATE EXTENSION postgis;`), donnant PostGIS 3.6 avec GEOS et PROJ actifs. Un schéma dédié, `fibre`, a été créé afin d'isoler les tables du projet, dans un cas d'usage de réseau fibre optique (FTTx).

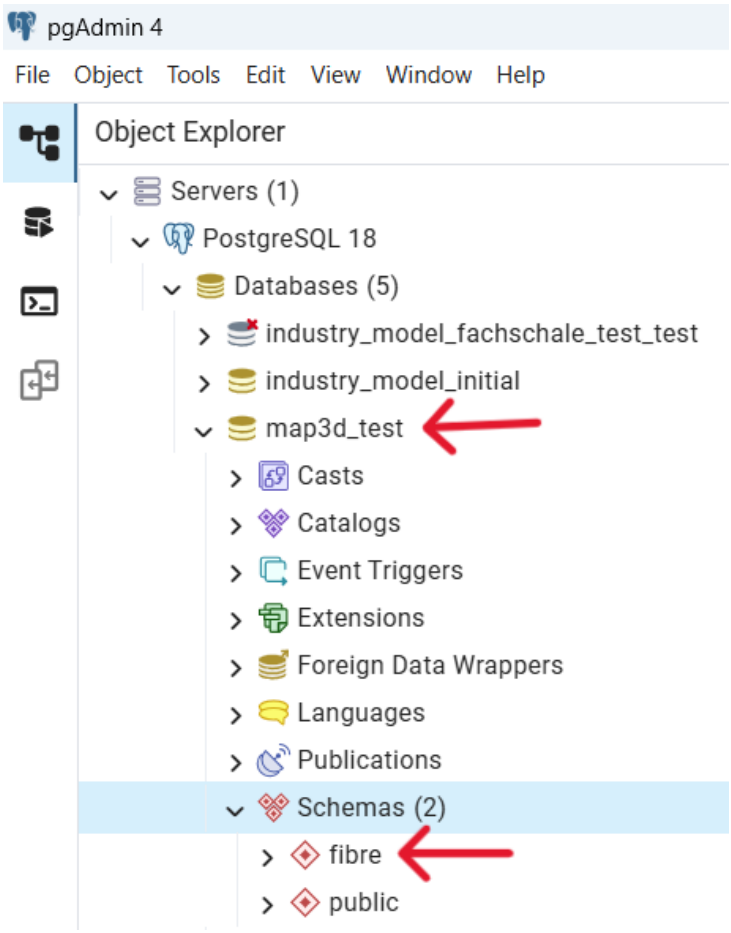


FIGURE 6 – Base de données de test : map3d_test

3.2 Schéma de démonstration

Un script SQL autonome (`demo_schema_fibre.sql`) a été rédigé afin de pratiquer manuellement la modélisation spatiale avant de chercher, en phase suivante, à la générer automatiquement à partir d'un Data Model Autodesk. Il crée, dans le schéma `fibre`, quatre tables spatiales : `chambre` et `poteau` en géométrie POINT, `cable` en LINESTRING avec clés étrangères vers `chambre`, et `zone_desserte` en POLYGON, avec index GiST et requêtes de test (longueur d'un câble, inclusion spatiale).

3.3 Test du connecteur FDO natif

Le connecteur natif *Bring in features from PostgreSQL/PostGIS* de Map 3D a été testé avec succès. La connexion à la base `map3d_test` a été établie via *Data Connect* → *Add PostgreSQL Connection*, avec authentification sur le service local. Une fois connecté, l'arborescence du schéma `fibre` apparaît avec les tables spatiales créées, chacune reconnue avec son système de coordonnées.

La table `chambre` a été ajoutée au dessin, avec ses enregistrements affichés correctement (identifiant, nom, type). Un test de modification a ensuite été réalisé : le nom d'un enregistrement a été modifié directement dans Map 3D, puis vérifié dans pgAdmin via une requête `SELECT`. Les modifications effectuées depuis Map 3D se sont révélées correctement répercutées dans PostgreSQL, géométrie comprise, confirmant un fonctionnement bidirectionnel sans étape de commit explicite visible.

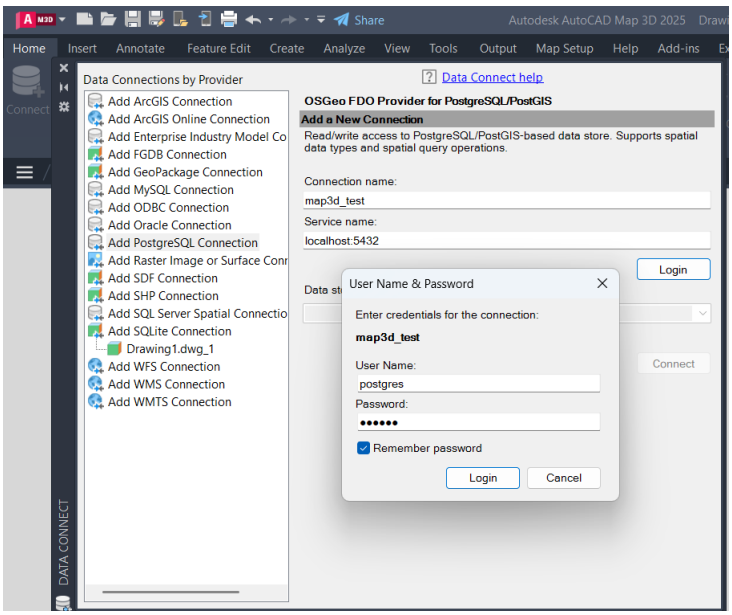


FIGURE 7 – Fenêtre de connexion PostgreSQL — saisie des identifiants pour la base map3d_test

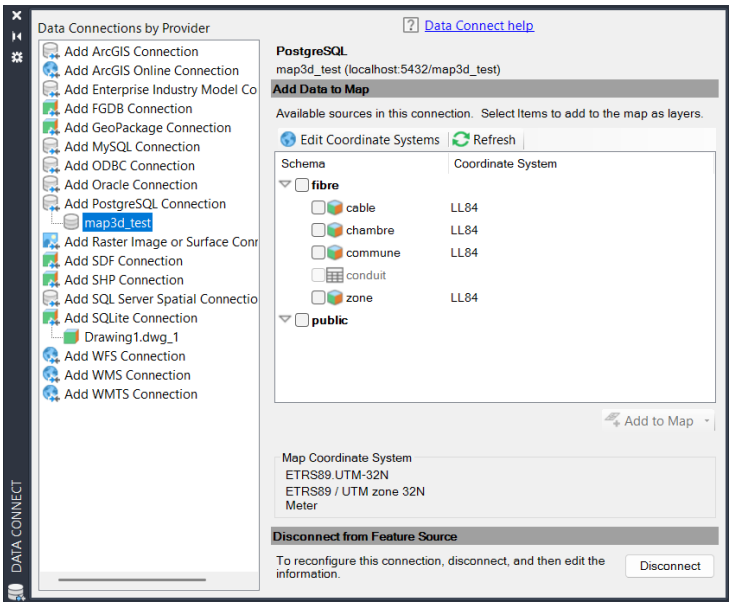


FIGURE 8 – Panneau « Add Data to Map » — sélection des couches du schéma fibre à ajouter au dessin

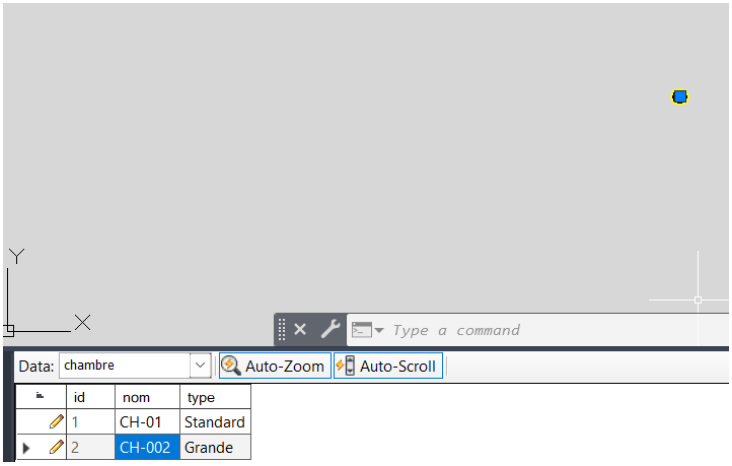


FIGURE 9 – Sélection de l'objet 'CH-002' dans le dessin — correspondance confirmée avec la base

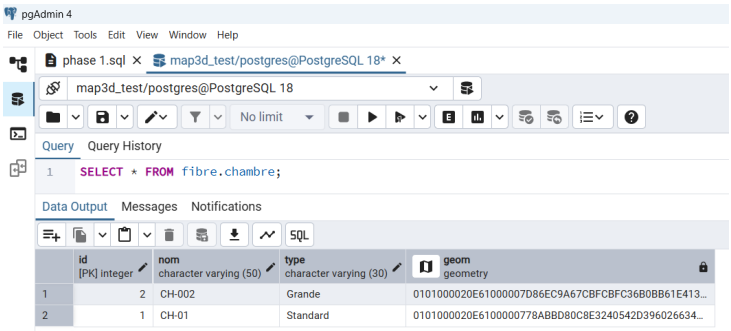


FIGURE 10 – Vérification en base via pgAdmin — les attributs modifiés dans Map 3D sont bien persistés dans fibre.chambre

3.4 Ce que le connecteur FDO apporte – et n'apporte pas

Le fournisseur FDO joue uniquement le rôle de connecteur entre AutoCAD Map 3D et la base de données : il ouvre une connexion, lit les tables contenant des géométries, les affiche comme couches cartographiques, et permet leur création, modification et suppression, avec enregistrement automatique dans PostgreSQL. Chaque table PostgreSQL est alors simplement interprétée comme une couche cartographique.

Le connecteur FDO ne possède en revanche aucune connaissance du métier représenté par ces objets. Il ne fournit ni les classes métier d'une Fachschale, ni les règles métier, ni les formulaires spécialisés, ni les relations métier entre objets, ni les métadonnées propres aux Industry Models Autodesk. Autrement dit, FDO permet uniquement l'accès aux données géographiques : il ne fournit pas la couche métier qui caractérise une Fachschale.

Cette observation permet de positionner plus précisément TKI PGP : sa valeur ajoutée ne se situe pas dans la connexion à PostgreSQL, déjà assurée nativement par FDO, mais dans l'apport des éléments qui dépassent la simple lecture de tables spatiales – une hypothèse qui sera vérifiée au Chapitre 5.

3.5 Conclusion de la phase

Les essais réalisés montrent qu'AutoCAD Map 3D peut communiquer directement avec une base PostgreSQL/PostGIS grâce au fournisseur **OSGeo FDO Provider for PostgreSQL/PostGIS** qui lui est intégré, sans recourir au connecteur commercial TKI PGP, pour tout ce qui relève de données SIG classiques. En revanche, une simple connexion FDO ne suffit pas à reproduire le comportement complet d'une Fachschale Autodesk. L'étape suivante du projet consiste donc à comprendre comment un Industry Model est défini et transformé en Data Model, ce qui fait l'objet du chapitre suivant.

Chapitre 4: Rétro-ingénierie du Data Model Autodesk

Ce chapitre détaille la méthodologie de rétro-ingénierie différentielle mise en œuvre pour comprendre la structure interne du Data Model Autodesk. Cette étape constitue l'une des parties les plus critiques et les plus chronophages du projet, en raison de l'absence d'une documentation explicite décrivant précisément la correspondance entre les éléments du modèle conceptuel et leur représentation physique dans les fichiers SQLite. La compréhension de cette correspondance a ainsi nécessité une démarche expérimentale rigoureuse, fondée sur des modifications contrôlées, des comparaisons systématiques et l'analyse des changements observés. Le chapitre présente la campagne de tests, les résultats obtenus et l'identification des six catalogues maîtres qui constituent la clé de la conversion automatique.

4.1 Objectif et enjeux de la phase

Le projet global vise à produire un outil capable de lire un Data Model Autodesk et de générer automatiquement un schéma PostgreSQL/PostGIS strictement équivalent. Un tel convertisseur ne peut être conçu correctement que si l'on comprend exactement comment Autodesk traduit, en interne, les concepts manipulés dans l'interface graphique (classes, attributs, domaines, relations, représentations graphiques) en structures relationnelles concrètes. Cette phase constitue donc la fondation technique de l'ensemble du projet : toute erreur d'interprétation à ce stade se répercuterait mécaniquement sur la génération du schéma, et le mapping conceptuel construit ici constitue la spécification fonctionnelle du futur moteur de conversion.

Il est essentiel de distinguer deux couches décrivant le même Data Model : la **couche conceptuelle**, visible et manipulable dans Infrastructure Administrator (arborescence des classes, propriétés, domaines, relations), et la **couche physique**, sa représentation réelle persistée dans un fichier SQLite. C'est cette dernière qui devra être lue par le futur outil de conversion, et l'objectif de cette phase est de construire, empiriquement et avec preuves, la table de correspondance entre ces deux couches.

4.2 Méthodologie

La méthode retenue est celle du **différentiel contrôlé** : chaque test consiste à effectuer une seule modification dans Infrastructure Administrator, puis à comparer l'état du schéma SQLite avant et après cette modification. Le cycle opératoire comprend systématiquement la modification dans Infrastructure Administrator, la sauvegarde permettant de persister les changements dans SQLite, l'extraction des fichiers `.sql` avant et après modification, le **lancement du script de comparaison** (`compare_sqlite.py`), puis l'analyse des différences observées afin d'identifier le rôle fonctionnel des tables et des colonnes concernées. En isolant ainsi chaque modification, cette approche permet d'établir de manière reproductible la correspondance entre le modèle conceptuel et sa représentation physique, sans attribuer un changement à plusieurs causes potentielles. Cette contrainte constitue le principe fondamental du protocole expérimental.

4.3 Fonctionnement du script de comparaison automatisée

L'étape « Lancement du script de comparaison » du cycle opératoire repose sur un outil Python développé spécifiquement pour ce projet, `compare_sqlite.py`. Cet outil constitue en lui-même un test à part entière de la campagne (Test 19), au sens où sa conception a nécessité de comprendre finement les pièges d'un export SQLite Autodesk avant de pouvoir l'exploiter de façon fiable.

Une comparaison naïve des fichiers `.sql` ligne à ligne, à la manière d'un `diff` Unix classique, a été volontairement écartée. En effet, un export SQLite n'est pas nécessairement stable dans sa représentation textuelle : un simple réordonnancement de lignes, une réécriture équivalente d'un `CREATE TABLE` ou une différence de formatage des valeurs numériques pourrait produire de nombreux faux positifs, sans que la structure logique du Data Model ait réellement changé.

Le script délègue donc l'interprétation du SQL au moteur SQLite lui-même. Chaque export est rechargé dans une base temporaire en mémoire , puis interrogé à l'aide des mécanismes internes du moteur, notamment la table système `sqlite_master`, qui permet d'inventorier les tables, index, triggers et vues, ainsi que les directives `PRAGMA table_info` et `PRAGMA foreign_key_list`. La première permet d'obtenir les informations relatives aux colonnes, aux types, aux contraintes `NOT NULL`, aux valeurs par défaut et aux clés primaires. La seconde permet d'identifier les relations entre les classes.

La comparaison porte ainsi sur la **structure logique** des objets SQLite plutôt que sur leur simple représentation textuelle, ce qui permet d'identifier de manière plus fiable les modifications réellement induites par chaque opération effectuée dans Infrastructure Administrator.

4.3.1 Les grandes étapes du traitement

Le tableau suivant résume les fonctions principales du script et le rôle exact de chacune dans la chaîne de traitement :

TABLE 1 – Fonctions principales de `compare_sqlite.py`

Fonction	**Rôle**
`split_sql_statements`	Découpe le script SQL en instructions individuelles séparées par `;`, en respectant les chaînes de caractères (`'...'`, `"..."`) et surtout les blocs `BEGIN...END` / `CASE...END` des triggers, qui contiennent des `;` internes ne devant pas être interprétés comme des fins d'instruction.
`normalize_special_floats`	Réécrit les jetons non quotés `Inf`, `-Inf`, `NaN` (fréquents dans les coordonnées d'emprise FDO non bornées) en littéraux texte valides, afin d'éviter un échec silencieux de l'`INSERT` correspondant.
`safe_executescript`	Exécute le script instruction par instruction plutôt qu'en un seul bloc, afin qu'une instruction non supportée (table déjà existante, syntaxe propriétaire) soit simplement ignorée et journalisée, sans faire échouer tout le chargement.
`load_schema`	Charge un export `schema_testXX.sql` en mémoire et en extrait un `SchemaSnapshot` structuré : tables, colonnes (avec types et contraintes), clés étrangères, index, triggers et vues.
`load_dump`	Charge un export `dump_testXX.sql` et reconstitue le contenu de chaque table sous forme de dictionnaire de lignes indexées par clé primaire (ou par la ligne entière si aucune clé n'est définie).
`compare_schemas`	Compare deux `SchemaSnapshot` et produit la liste des tables, colonnes, clés étrangères, index, triggers et vues ajoutés, supprimés ou modifiés.
`compare_dumps`	Compare deux jeux de données **par clé** (et non par simple appartenance à un ensemble), ce qui permet de distinguer une ligne ajoutée, une ligne supprimée et une ligne modifiée (UPDATE implicite).
`generate_report`	Assemble les deux diffs précédents en un rapport Markdown structuré et directement exploitable comme preuve reproductible dans ce mémoire.
`run_batch` / `discover_test_files`	Mode automatisé : parcourt récursivement un répertoire, associe chaque paire `schema_testN`/`dump_testN`, compare automatiquement chaque test au précédent, et génère un rapport de synthèse global (`report_summary.md`).

4.3.2 Un point technique déterminant : la gestion des blocs de trigger

La difficulté la plus significative rencontrée lors du développement de l'outil concerne le découpage des instructions SQL en présence de *triggers*. Comme le montre le Test 1, chaque classe créée génère cinq triggers SQLite (gestion de l'identifiant `FID`), dont le corps contient lui-même des points-virgules internes :

```
CREATE TRIGGER trg_check BEFORE INSERT ON tb_class
BEGIN
    SELECT CASE WHEN NEW.FID IS NULL
        THEN RAISE(ABORT, 'FID required') END;
```

```
    UPDATE tb_class SET x = NEW.FID WHERE id = NEW.id;
END;
```

Un découpage naïf sur chaque `;` romprait cette instruction en trois fragments distincts, dont l'exécution isolée provoquerait des erreurs en cascade (`RAISE()` utilisé hors contexte de trigger, colonne `NEW.FID` inconnue). Le script maintient donc un compteur de profondeur de bloc (`block_depth`), incrémenté à chaque `BEGIN` ou `CASE` et décrémenté à chaque `END` rencontré, et n'autorise une coupure sur `;` que lorsque ce compteur revient à zéro. Une attention particulière est également portée à la distinction entre le `BEGIN` d'un trigger (qui se termine par `END`) et le `BEGIN TRANSACTION` d'une transaction SQL classique (qui se termine par `COMMIT`/`ROLLBACK`), afin que ce dernier ne soit jamais compté comme un bloc ouvrant.

4.3.3 Sortie produite

Pour chaque transition entre deux tests, l'outil produit un rapport Markdown reprenant systématiquement les mêmes sections : tables ajoutées/supprimées, colonnes ajoutées/-supprimées/modifiées, relations (clés étrangères) ajoutées/supprimées, index/triggers/-vues ajoutés/supprimés, puis le détail des données (lignes ajoutées/supprimées/modifiées par table). En mode batch, un rapport de synthèse global agrège ces informations sous forme d'un tableau récapitulatif chiffré (nombre de tables, colonnes, clés étrangères et lignes affectées par transition), ce qui a permis de vérifier rapidement, à chaque test, qu'aucune table inattendue n'avait été modifiée en dehors du périmètre de la seule action réalisée – condition indispensable au respect de la règle de modification unique.

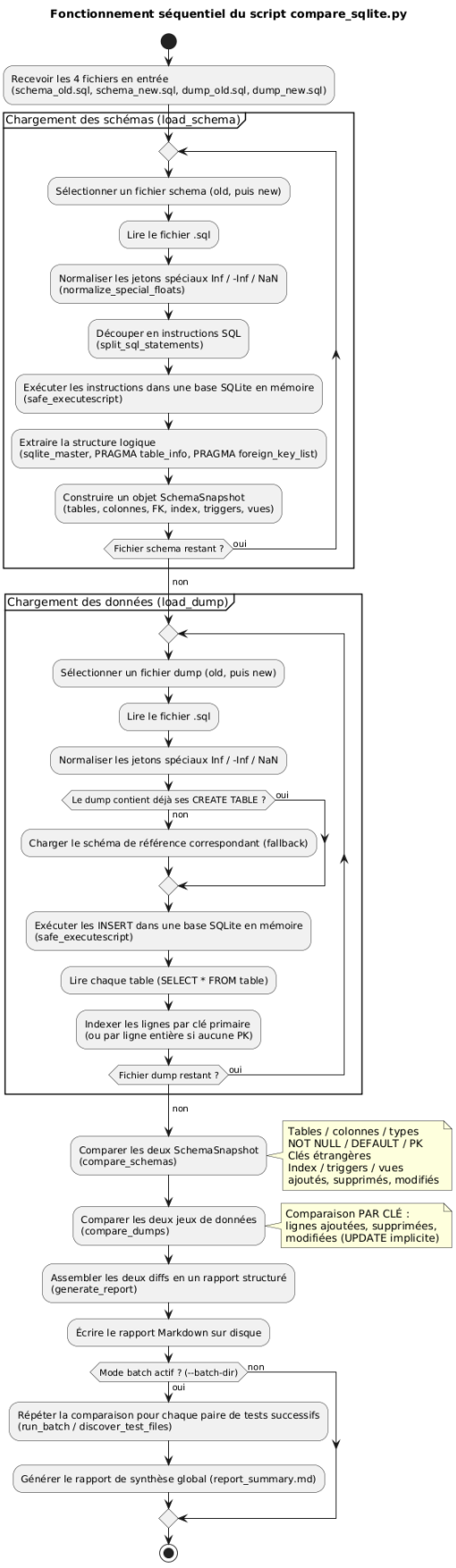


FIGURE 11 – Diagramme d'activité illustrant le fonctionnement séquentiel du script de comparaison des tests

4.4 Campagne de tests

Une campagne de dix-huit tests contrôlés a été menée sur un Data Model de test unique, en partant d'un état de référence puis en isolant successivement chaque concept structurel : classes non géométriques et géométriques, attributs et leurs contraintes, relations, domaines de valeurs, héritage, renommage et suppressions. Les paragraphes suivants détaillent les résultats obtenus pour chaque famille de concepts ; le tableau de suivi complet figure en section **??**.

4.4.1 État de référence (Test 0)

Avant toute modification, un état baseline a été établi afin de disposer d'un point de comparaison pour l'ensemble des tests suivants. Le Data Model vierge comporte 170 tables au total, majoritairement des tables système Autodesk préfixées `TB_`, ainsi que les tables standard OGC (`geometry_columns`, `spatial_ref_sys`). Cette référence confirme qu'un Data Model « vide » n'est jamais réellement vide : il embarque dès l'initialisation l'intégralité de l'infrastructure de métadonnées nécessaire au moteur FDO.

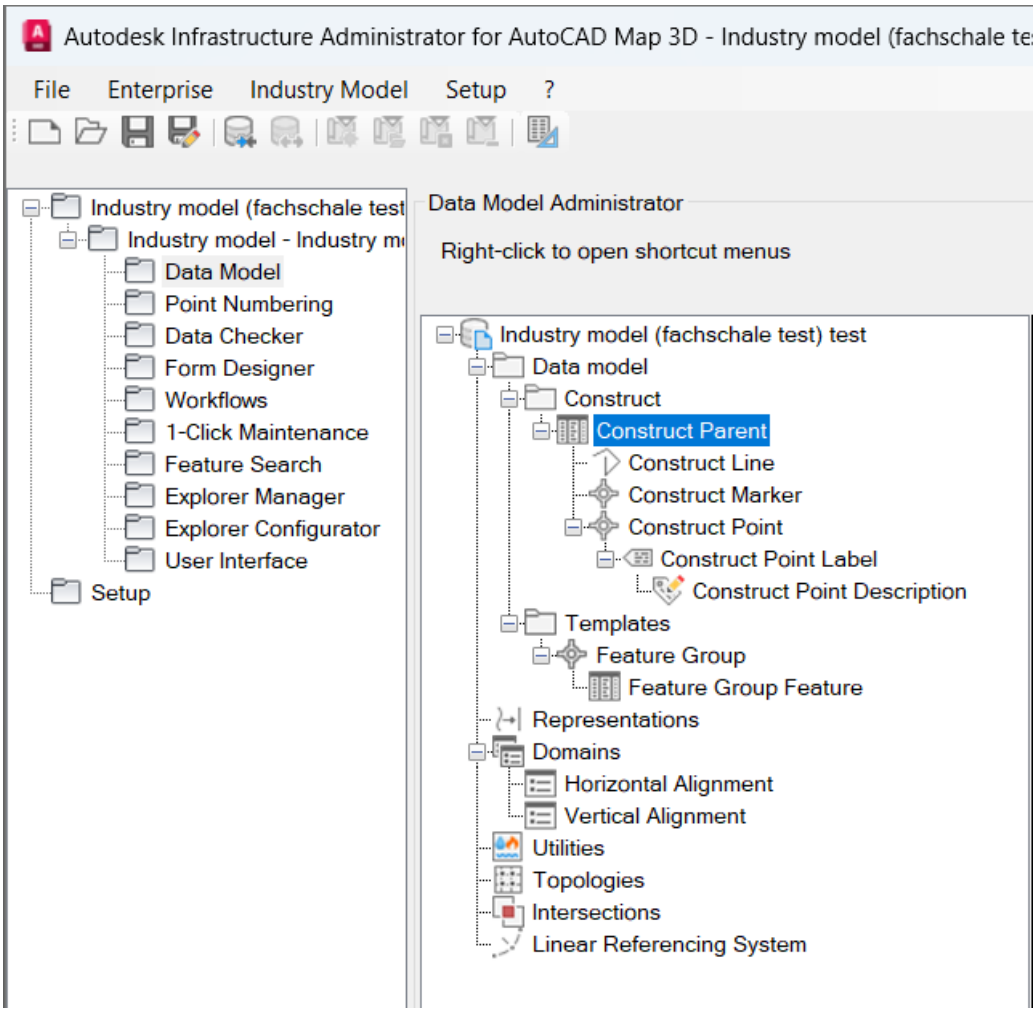


FIGURE 12 – Arborescence complète du Data Model dans Infrastructure Administrator

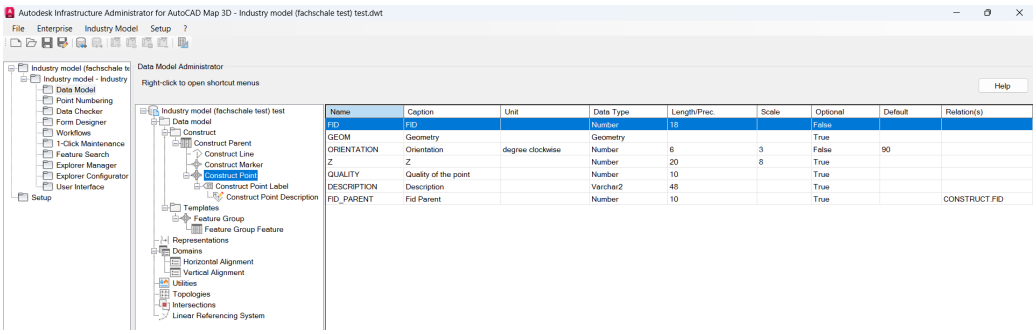


FIGURE 13 – Liste des attributs d'une classe (exemple de la classe `Construct Point`)

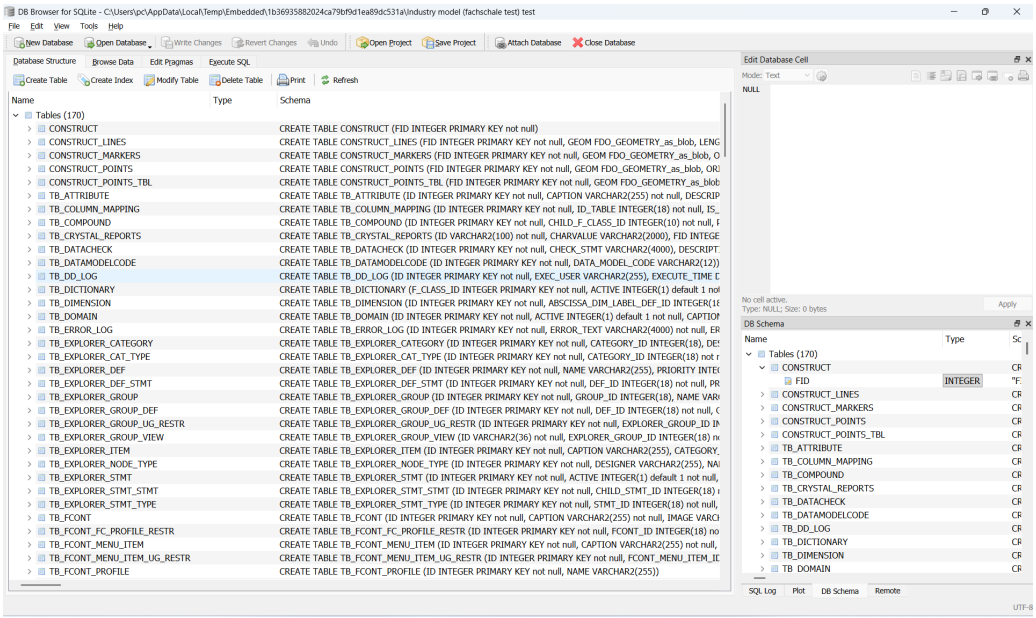


FIGURE 14 – État initial du fichier SQLite dans DB Browser for SQLite – 170 tables présentes par défaut

4.4.2 Création de classes (Tests 1, 7 et 8)

La création d'une classe non géométrique (Test 1) déclenche la création immédiate d'une véritable table physique SQLite, accompagnée de cinq triggers automatisant la gestion de l'identifiant (`_AD_FID`, `_AI_FID`, `_AU_FID`, `_BI_FID`, `_BU_FID`). Au niveau catalogue, une ligne est ajoutée dans `TB_DICTIONARY` (définition de la classe), une ligne dans `TB_ATTRIBUTE` et `fdo_columns` (déclaration de la clé primaire `FID`), et six lignes de comportements dans `TB_RULE_BASE`. Une vingtaine de lignes supplémentaires sont injectées dans les tables d'interface (`TB_GN_*`, `TB_SETTINGS`), sans impact sur la structure relationnelle des données.

Les Tests 7 et 8 reproduisent cette opération pour des classes géométriques de type Point et Ligne. Dans les deux cas, une ligne supplémentaire apparaît dans `geometry_columns`, déclarant la colonne physique `GEOM` avec un `geometry_type` respectivement égal à 1 (Point) et 2 (LineString). Le champ `F_CLASS_TYPE` de `TB_DICTIONARY` encode le même distinguo (`'P'` vs `'L'`). Chaque type géométrique génère par ailleurs son propre sous-ensemble d'attributs techniques : `Z`, `ORIENTATION` et `QUALITY` pour un point, contre un unique attribut calculé `LENGTH` (précision 20, échelle 8) pour une ligne.

4.4.3 Attributs et contraintes (Tests 2 à 6)

L'ajout d'un attribut, quel que soit son type, se traduit systématiquement par un `ALTER TABLE` sur la table métier parente, associé à une ligne dans `TB_ATTRIBUTE` (rattachement à la classe mère via `F_CLASS_ID`) et une ligne dans `fdo_columns` portant le typage FDO exact. Le Test 2 (attribut texte) enregistre un `fdo_data_type` de 9 avec une longueur de 10, tandis que le Test 3 (attribut numérique) enregistre un `fdo_data_type` de 7 avec une précision de 10 – confirmant qu'**aucune table de référence des types n'existe** : le typage est directement formalisé en SQL natif dans le DDL de la table (`VARCHAR2(10)`, `INTEGER(10)`) et redondé sous forme de code numérique dans `fdo_columns`.

Les Tests 4 et 5 montrent qu'Infrastructure Administrator ne stocke aucune métadonnée logicielle propre pour les contraintes de valeur par défaut et de champ obligatoire : ces règles sont directement traduites en clauses DDL natives du moteur SQLite (`DEFAULT 0`, `NOT NULL`) sur la colonne physique, sans trace additionnelle dans `TB_ATTRIBUTE` ou `fdo_columns`. Le Test 6, portant sur la longueur maximale d'un champ, a été jugé redondant et abandonné : ce comportement était déjà pleinement démontré par les Tests 1, 2 et 5.

4.4.4 Relations et domaines de valeurs (Tests 9 à 11)

Le Test 9 établit qu'une relation 1-N entre deux classes ne se limite pas à une ligne descriptive dans `TB_RELATIONS` : elle matérialise également une véritable colonne de clé étrangère physique dans la table enfant, déclarée dans `TB_ATTRIBUTE` et `fdo_columns`, et accompagnée d'un index dédié. Aucune cardinalité (`MERGE_MODE` / `SPLIT_MODE`) n'a cependant été observée dans ce test simple.

Les Tests 10.1 et 10.2 montrent qu'un domaine de valeurs (liste déroulante) repose sur un mécanisme distinct mais analogue : sa création (Test 10.1) génère une ligne catalogue dans `TB_DOMAIN` et une table physique dédiée portant son nom (par ex. `TEST_DOMAINE_10_TBD`) contenant les valeurs autorisées. Le rattachement d'un attribut à ce domaine (Test 10.2) réutilise *exactement* le mécanisme de relation du Test 9 : une colonne FK est ajoutée à la classe enfant, et une ligne `TB_RELATIONS` pointe cette fois vers la table de domaine plutôt que vers une classe métier – confirmant qu'un domaine est traité en interne comme une « classe parente » au sens relationnel. Le Test 11 confirme enfin que l'ajout d'une valeur à un domaine existant (`Cuivre`) se limite à un simple `INSERT` incrémental dans la table de valeurs, sans aucun impact sur le catalogue `TB_DOMAIN`.

4.4.5 Héritage entre classes (Test 12)

La création d'une classe fille héritant d'une classe parente révèle un mécanisme plus profond qu'une simple référence logique. La table fille est créée avec sa structure complète, puis **recopie physiquement** l'ensemble des colonnes héritées – y compris celles issues de relations ou de domaines, elles-mêmes redéclarées dans `TB_RELATIONS`. Le lien parent-enfant est porté par le champ `MODEL_F_CLASS_ID` de `TB_DICTIONARY`, et non par `PARENT_F_CLASS_ID` (qui référence une classe système générique). La classe parente reçoit, de son côté, un attribut `MODEL_NAME` la qualifiant de modèle réutilisable.

4.4.6 Renommage et suppression (Tests 16 à 18)

Le Test 16 démontre que le renommage d'une classe est une opération purement cosmétique : seules les colonnes `CAPTION` et `FEATURE_REPRESENTATION` de `TB_DICTIONARY` sont modifiées, tandis que l'identifiant technique `F_CLASS_ID` et le nom physique de la table SQLite restent parfaitement stables – un point rassurant pour la fiabilité du futur mapping, puisqu'il découple le nom métier affiché de la structure technique sous-jacente.

Les Tests 17 et 18 confirment un comportement de **suppression physique totale (hard delete)**, symétrique à la création, à deux échelles différentes. La suppression d'une classe (Test 17) déclenche une cascade complète : suppression de `TB_DICTIONARY`, `TB_ATTRIBUTE`, `TB_RELATIONS`, `TB_RULE_BASE` et `fdo_columns`, ainsi qu'un `DROP TABLE` de la table physique, de ses index et de ses triggers, sans laisser de résidu. La suppression d'un simple attribut (Test 18) est plus ciblée : elle produit un véritable `DROP COLUMN` sur la table parente, avec nettoyage en cascade de `TB_ATTRIBUTE` et `fdo_columns`, sans toucher aux index ni aux triggers (rattachés à la classe et non à l'attribut). Dans les deux cas, **aucun marquage logique** (`deleted = 1`) n'a été observé : Autodesk ne pratique jamais de soft delete.

4.5 Résultat : les six catalogues maîtres

L'analyse croisée de l'ensemble des tests a permis d'identifier, parmi la centaine de tables générées par Infrastructure Administrator sur le modèle de test (la majorité étant des tables internes de configuration UI), six catalogues maîtres qui semblent porter l'intégralité de l'information structurelle nécessaire à la reconstruction du Data Model :

TABLE 2 – Les six catalogues maîtres du Data Model Autodesk

Concept	**Table(s) maîtresse(s)**	**Mécanisme observé**
Classe	`TB_DICTIONARY`	Nom/type de classe (Point, Ligne, Table), création de la table physique
Attribut	`TB_ATTRIBUTE`, `fdo_columns`	Type FDO, longueur, précision
Géométrie	`geometry_columns`	Type de géométrie OGC (1=Point, 2=Ligne)
Domaine (liste déroulante)	`TB_DOMAIN`, `<DOMAINE>_TBD`	Table de valeurs dédiée
Relation	`TB_RELATIONS`	Colonne FK dans la table enfant
Héritage	`TB_DICTIONARY` (`MODEL_F_CLASS_ID`)	Copie physique complète des colonnes vers la table fille

4.6 Catégorisation des tables SQLite

Les tables observées ont pu être réparties en quatre catégories, ce qui permet de distinguer clairement ce qui doit être repris par le futur convertisseur de ce qui relève de la seule

configuration interne d'Autodesk :

TABLE 3 – Catégorisation des tables SQLite observées

Catégorie	**Description**	**Exemples**
Tables métier	Définitions utilisateur (classes, domaines), à générer en PostgreSQL/PostGIS	Tables de classes et de domaines créées pendant les tests
Tables système Autodesk (catalogues)	Structures logiques (métadonnées) du modèle gérées par l'API FDO	`TB_DICTIONARY`, `TB_ATTRIBUTE`, `TB_RELATIONS`, `TB_DOMAIN`, `fdo_columns`, `geometry_columns`
Tables de configuration/interface	Paramètres UI, sans impact sur la structure des données métier	`TB_GN_FLYIN_USER`, `TB_SETTINGS`
Générateurs / séquences	Simulation d'identifiants séquentiels	`TB_SEQUENCE_EMULATION`, triggers générés

Il convient de noter qu'une suppression de classe ou d'attribut déclenche systématiquement un nettoyage en cascade total de ces catégories métier et catalogue (hard delete), une propriété qu'il faudra reproduire sous PostgreSQL au moyen de contraintes `ON DELETE CASCADE`.

4.7 Périmètre exclu de l'analyse

Quatre tests menés en cours de campagne (6, 13, 14 et 15) ont été volontairement exclus de l'analyse structurelle, au regard de l'objectif fondamental de cette phase : déterminer quelles tables, colonnes, contraintes et clés doivent être créées pour générer automatiquement un schéma PostgreSQL/PostGIS équivalent.

- Le **Test 6** (longueur maximale d'un champ) a été jugé redondant, son comportement ayant déjà été pleinement démontré par le croisement des Tests 1, 2 et 5 (ex. `Varchar2(10)`) ;
- Le **Test 13** (représentation graphique) ne concerne que la stylisation SIG (couleurs, épaisseurs, symboles) : PostgreSQL n'a pas à savoir si une classe spatiale doit s'afficher en rouge ou en pointillé, ce style relevant de l'application cliente (Map 3D, QGIS) ;
- Le **Test 14** (labels) représente uniquement l'étiquetage affiché sur la carte (ex. afficher le champ « ID » plutôt que « Diamètre ») ;
- Le **Test 15** (templates de saisie) porte sur les assistants d'auto-remplissage des champs, une aide à l'interface sans effet structurel.

Ces quatre fonctionnalités relèvent de la couche de présentation ou de l'assistance à la saisie et n'ont, par construction, aucun impact sur la structure relationnelle des données SQL sous-jacentes.

4.8 Conclusion

Cette phase de rétro-ingénierie permet de comprendre et de documenter intégralement le fonctionnement interne du Data Model Autodesk, tel qu'il est physiquement stocké dans le fichier SQLite sous-jacent à Infrastructure Administrator. Elle fournit une cartographie tracée des tables et colonnes concernées, un ensemble de preuves reproductibles pour chacun des dix-huit tests menés, une classification claire entre tables métier et tables système, ainsi que des spécifications fonctionnelles robustes pour la structure des géométries, des héritages et des contraintes. Il suffira, pour le futur convertisseur automatisé, de lire les six catalogues maîtres identifiés – au premier rang desquels `TB_DICTIONARY`, `TB_ATTRIBUTE` et `TB_RELATIONS` – pour collecter l'essentiel du modèle nécessaire à sa transformation en script DDL PostgreSQL/PostGIS. Aucune métadonnée cachée intraçable n'a été isolée à ce stade. Cette base permet d'aborder sereinement la conception du moteur de conversion automatique, cœur des chapitres suivants.

Chapitre 5: Rôle de TKI PGP dans l'architecture Autodesk

Ce chapitre analyse le connecteur commercial TKI PGP à partir de sa documentation et de son comportement observable, afin d'en dégager les fonctions clés que la solution alternative devra reproduire. Il compare également les différentes approches de persistance (Oracle, SQL Server, PostgreSQL via TKI) et pose les questions ouvertes.
***NB :** Aucune information relative à la licence ou au code source du connecteur TKI PGP n'a été mise en évidence dans le cadre de cette étude. L'analyse présentée repose principalement sur la documentation disponible ainsi que sur des hypothèses formulées à partir des fonctionnalités observées et des résultats obtenus lors des expérimentations.*

5.1 Présentation

TKI PGP (*PostgreSQL Provider*) est un logiciel de connectivité développé par TKI Software / TKI Chemnitz, destiné à permettre l'utilisation des Industry Models Autodesk avec PostgreSQL plutôt qu'avec Oracle ou SQL Server. La documentation officielle le présente comme le connecteur effectif entre AutoCAD Map 3D et PostgreSQL. Il s'agit d'un produit commercial et versionné, nécessitant une licence activée sur un serveur de licences réseau, avec une compatibilité stricte de version vis-à-vis d'AutoCAD Map 3D. Sa documentation indique qu'il prend en charge les fonctionnalités de l'Infrastructure Administrator, l'import/export de dumps PostgreSQL, ainsi que le SQL sheet de Map 3D.

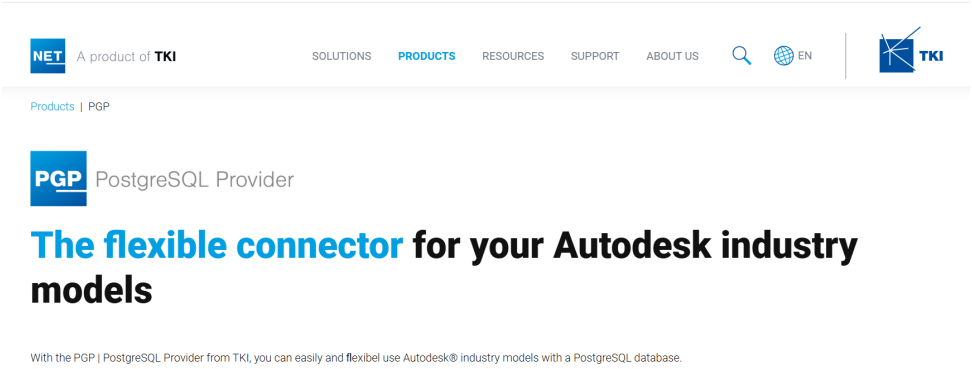


FIGURE 15 – Site Officiel du TKI Engineering

5.2 Analyse fonctionnelle

TKI PGP s'insère dans l'architecture Autodesk comme une couche de liaison spécialisée entre les Industry Models et une base PostgreSQL. Il ne se limite pas à ouvrir une connexion : il adapte le fonctionnement du modèle Autodesk à l'environnement PostgreSQL, en faisant correspondre les attentes du monde Autodesk aux capacités de PostgreSQL. Il joue ainsi un rôle intermédiaire entre l'application Autodesk, la logique métier des Industry Model, et le SGBD PostgreSQL chargé de la persistance.

Cette analyse permet de dégager une synthèse fonctionnelle : PGP relie les Industry Model Autodesk à PostgreSQL, conserve les workflows d'administration déjà utilisés dans l'environnement Autodesk, facilite l'échange de données via les dumps, et permet l'exploitation

opérationnelle via le SQL sheet. Il agit ainsi comme un véritable **pivot fonctionnel** : il ne remplace ni Autodesk ni PostgreSQL, mais rend les deux compatibles dans un cadre métier précis.

5.3 Questions ouvertes

Certains points n'ont pas pu être confirmés par une source officielle et restent, à ce stade, des hypothèses de travail : le mécanisme exact de génération du schéma PostgreSQL par PGP (création complète ou réplication d'une structure Oracle/SQL Server existante), l'existence éventuelle d'un mécanisme de verrouillage multi-utilisateur analogue au check-out/check-in, la réutilisation ou non du connecteur FDO natif en interne, et la prise en charge effective des règles métier et des formulaires. Ces questions ont été transmises telles quelles à la phase de conception de l'architecture cible, afin d'orienter les choix techniques.

5.4 Synthèse et pistes pour la solution alternative

Cette analyse met en évidence quatre fonctions clés que la solution alternative doit reproduire à un niveau équivalent : la génération et la synchronisation du schéma PostgreSQL/-PostGIS, l'intégration aux workflows de l'Infrastructure Administrator, l'import/export de données, et l'accès en lecture/écriture depuis Map 3D. La conception détaillée de cette solution fait l'objet du chapitre suivant.

Chapitre 6: Architecture cible et conception de la solution alternative

Ce chapitre présente la démarche d'ingénierie ayant conduit à la définition de l'architecture cible open-source. Il détaille l'étude comparative des cinq approches envisagées, la justification de l'architecture hybride retenue (A + E), ainsi que les spécifications algorithmiques complètes du moteur de conversion DDL Python.

6.1 Objectif et exigences fonctionnelles

L'objectif de cette phase est de concevoir l'architecture technique capable d'exporter et d'exploiter un Data Model Autodesk sous PostgreSQL/PostGIS de manière 100% autonome, en éliminant la dépendance aux SGBD propriétaires (Oracle, SQL Server) et au connecteur commercial payant (TKI PGP).

La solution doit répondre à deux exigences fondamentales et indissociables :

1. **La génération automatisée et exacte du schéma DDL** : Traduction fidèle de l'intégralité des métadonnées du Data Model SQLite (classes, attributs, énumérations, relations, contraintes, index spatiaux, triggers) vers un script SQL PostGIS exécutable.
2. **L'exploitation cartographique temps réel et bidirectionnelle** : Capacité pour les géomaticiens d'éditer et de créer des objets sous AutoCAD Map 3D en direct sur la base PostgreSQL générée, sans phase d'import/export manuelle.

6.2 Étude comparative et évaluation des approches

Afin d'identifier la meilleure alternative à TKI PGP, cinq approches techniques distinctes ont été modélisées et évaluées selon des critères de faisabilité, de performance temps réel, de complexité de déploiement et de pérennité.

TABLE 4 – Évaluation comparative des approches techniques envisagées

Approche	Statut	Analyse technique et décision
A. Générateur DDL Python	**Retenue**	Script d'introspection lisant directement les métadonnées SQLite. Offre un contrôle total et exact sur le typage FDO, la résolution des clés étrangères, les index GiST et la génération de triggers PL/pgSQL.
B. Synchronisation par lots (ETL)	Éliminée	Synchronisation différée par scripts d'import/export. Éliminée car incompatible avec l'édition collaborative temps réel ; risque majeur de conflits de saisie et d'incohérence de données.
C. Plugin C# / .NET (API Map 3D)	Éliminée	Développement d'un complément DLL compilé via l'API ObjectARX/.NET d'Autodesk. Éliminée en raison de la complexité de développement et de l'exigence de maintenance/compilation sur chaque poste client.
D. Plugin Java (API)	Éliminée	Écosystème Autodesk exclusivement orienté .NET et C++. Aucune API Java officielle n'existe pour AutoCAD Map 3D, rendant cette voie techniquement irréalisable.
E. Connecteur FDO PostgreSQL Natif	**Retenue**	Exploitation du provider OSGeo FDO PostgreSQL natif d'AutoCAD Map 3D (`_MAPCONNECT`). Garantit l'édition cartographique temps réel sans aucun plugin tiers.

6.3 Architecture hybride retenue (Combinaison A + E)

Les approches **A** (générateur DDL) et **E** (connecteur FDO natif) ont été combinées pour constituer l'architecture cible du projet. La complémentarité entre ces deux éléments résout l'impasse identifiée au Chapitre 3 : le moteur Python fournit la couche de structure et de métadonnées métier (ce que FDO ne sait pas faire), tandis que le fournisseur FDO assure le transport des géométries et l'édition temps réel (ce que le moteur DDL n'a pas à gérer).

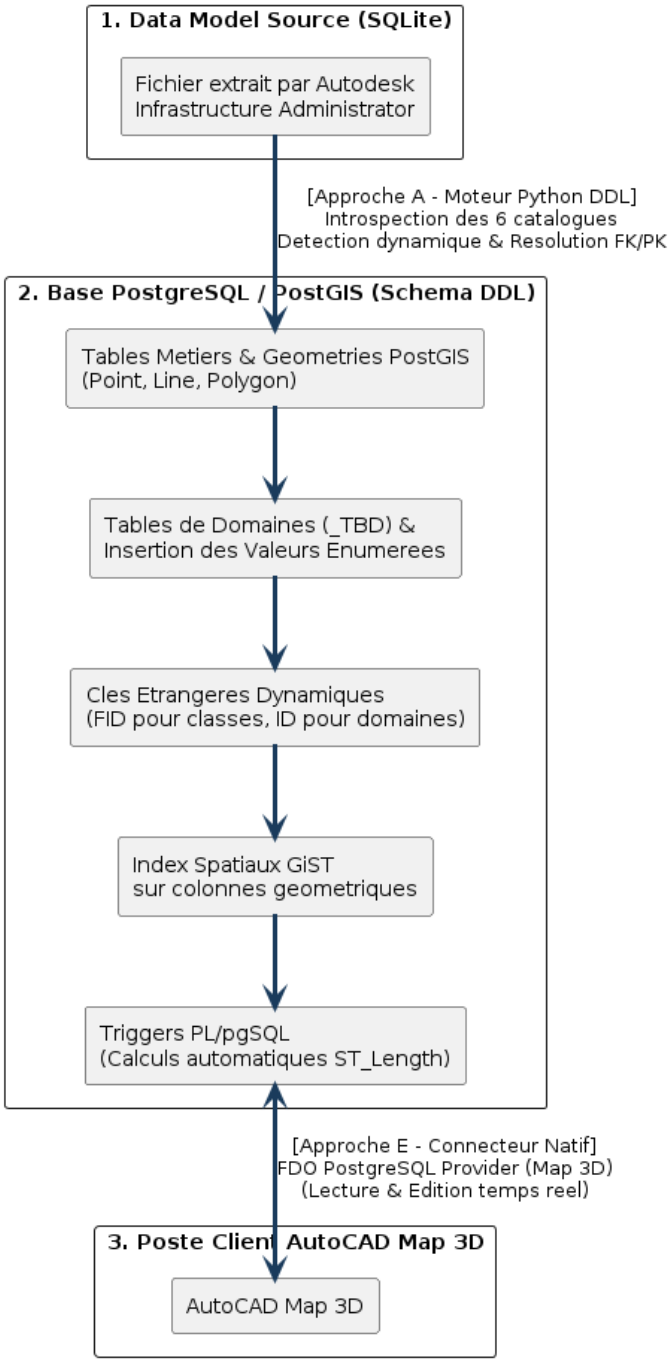


FIGURE 16 – Architecture cible de la solution : du Data Model Autodesk à AutoCAD Map 3D

6.4 Conclusion du chapitre

Ce chapitre a posé la conception formelle de la solution alternative. En combinant un moteur d'ingénierie DDL Python ciblé sur les métadonnées maîtres et l'exploitation directe du connecteur natif FDO d'AutoCAD Map 3D, la faisabilité théorique d'un remplacement de TKI PGP est démontrée. Le chapitre suivant présente le passage à l'échelle, l'implémentation opérationnelle, l'industrialisation sous forme d'application d'arrière-plan et la stack de monitoring.

Chapitre 7: Implémentation, synchronisation et validation du moteur

Ce chapitre présente l'implémentation logicielle du cœur de la solution. Il détaille en profondeur le moteur de conversion DDL (convert_autodesk_to_postgis.py), le moteur de surveillance temps réel (watch_and_sync.py), ainsi que l'ensemble des validations réalisées, tant par les tests unitaires automatisés (Pytest) que par l'exécution manuelle en ligne de commande.

7.1 Moteur de conversion DDL

Le moteur de conversion (`convert_autodesk_to_postgis.py`) constitue le cœur de la traduction de structure. Il réalise l'introspection directe du modèle de données SQLite exporté par Autodesk Infrastructure Administrator et génère un script SQL PostgreSQL/PostGIS complet.

7.1.1 Mapping des 6 catalogues maîtres vers PostgreSQL

Le script s'appuie sur la cartographie établie au Chapitre 4 pour traduire les métadonnées en structures relationnelles :

TABLE 5 – Mapping des six catalogues maîtres vers PostgreSQL

Catalogue maître	Traduction vers PostgreSQL
`TB_DICTIONARY`	Identifie le nom de la table métier (`F_CLASS_NAME`), son type d'entité (`F_CLASS_TYPE` : `'P'` pour Point, `'L'` pour Line, `'A'` pour Polygon, `'N'` pour Table attributaire) et l'identifiant de parenté (`MODEL_F_CLASS_ID`).
`TB_ATTRIBUTE`	Extrait la liste complète des attributs métiers configurés pour chaque classe d'objets.
`fdo_columns`	Restitue le typage FDO natif et effectue le mappage vers les types PostGIS équivalents (`Boolean` → `boolean`, `Int32/Int64` → `integer`, `Double` → `double precision`, `String` → `varchar(n)`, `DateTime` → `timestamp`).
`geometry_columns`	Associe la colonne spatiale (généralement `GEOM`) au type géométrique OGC PostGIS (`Point`, `LineString`, `Polygon`) en lui imposant le SRID cible de l'étude (ex : EPSG :2154 / Lambert-93).
`TB_DOMAIN` et tables `_TBD`	Extrait les tables de domaines de valeurs énumérées et génère les ordres `CREATE TABLE <DOMAINE>_TBD` suivis des insertions sécurisées (`INSERT ... ON CONFLICT DO NOTHING`).
`TB_RELATIONS`	Reconstruit l'intégralité des contraintes d'intégrité référentielle inter-classes et classe-domaine (`ALTER TABLE ... ADD CONSTRAINT FK_...`).

7.1.2 Résolution algorithmique des règles complexes

Le moteur de conversion intègre plusieurs mécanismes algorithmiques avancés pour garantir une traduction fidèle et robuste du Data Model Autodesk :

- **Tolérance aux variations de versions (`find_col_name()`)** : Afin de garantir la compatibilité entre les différentes versions d'Autodesk Infrastructure Administrator, la fonction inspecte dynamiquement le schéma physique des tables système SQLite (via `PRAGMA table_info`) pour identifier les colonnes réelles (ex : `f_class_id` vs `class_id` vs `id`).
- **Gestion de l'héritage et résolution des conflits de colonnes (`resolve_inheritance()`)** : Le moteur reconstitue l'arborescence d'héritage simple et multiple. En cas de conflit de nommage entre attributs hérités de plusieurs tables parentes, le script préfixe automatiquement la colonne par le nom de la table d'origine (ex : `PARENT_ID`) afin d'éviter tout échec lors du `CREATE TABLE`.
- **Résolution différenciée des clés d'intégrité (`get_pk_column_name()`)** : Le moteur distingue automatiquement les deux schémas de clés étrangères :
  - Clé `FID` lors d'une relation entre deux classes métier (`PARENT_TABLE` → `CHILD_TABLE`).
  - Clé `ID` lors d'un rattachement à une table de domaine énumérée (`<DOMAINE>_TBD`).
- **Documentation des métadonnées (`COMMENT ON CONSTRAINT`)** : Les paramètres de cardinalité et de comportement métier (*MergeMode*, *SplitMode*, *RelationType*) sont préservés sous forme de commentaires SQL sur les contraintes de clés étrangères.
- **Indexation spatiale et triggers PL/pgSQL** : Le moteur génère automatiquement un index spatial GiST pour chaque colonne géométrique (`CREATE INDEX ... USING GIST(geom)`) et déploie la fonction trigger PL/pgSQL `fn_calc_length_<table_name>` qui réévalue automatiquement la longueur des réseaux linéaires (`ST_Length`) à chaque écriture.

7.1.3 Fonctionnement séquentiel du moteur de conversion DDL

Le déroulement de l'exécution du script `convert_autodesk_to_postgis.py` s'effectue selon la séquence ordonnée suivante :

1. **Connexion à la base source** : ouverture d'une connexion SQLite standard.
2. **Lecture des catalogues de métadonnées** : extraction des classes métier et de leur héritage, des types FDO, des métadonnées spatiales, des relations et des tables de domaines.
3. **Génération des tables de domaines** : création et peuplement des tables de domaines énumérés.
4. **Génération des classes métier** : fusion des colonnes héritées, mapping des types FDO vers PostGIS, attribution de la géométrie (type OGC, SRID EPSG :2154) et création de l'index spatial GiST.
5. **Reconstitution de l'intégrité référentielle** : génération des contraintes de clés étrangères à partir de `TB_RELATIONS`.
6. **Génération des triggers PL/pgSQL** : calcul automatique des longueurs via `ST_Length` pour les classes linéaires concernées.
7. **Exportation du DDL** : assemblage du script SQL final, prêt à être exécuté sur PostgreSQL.

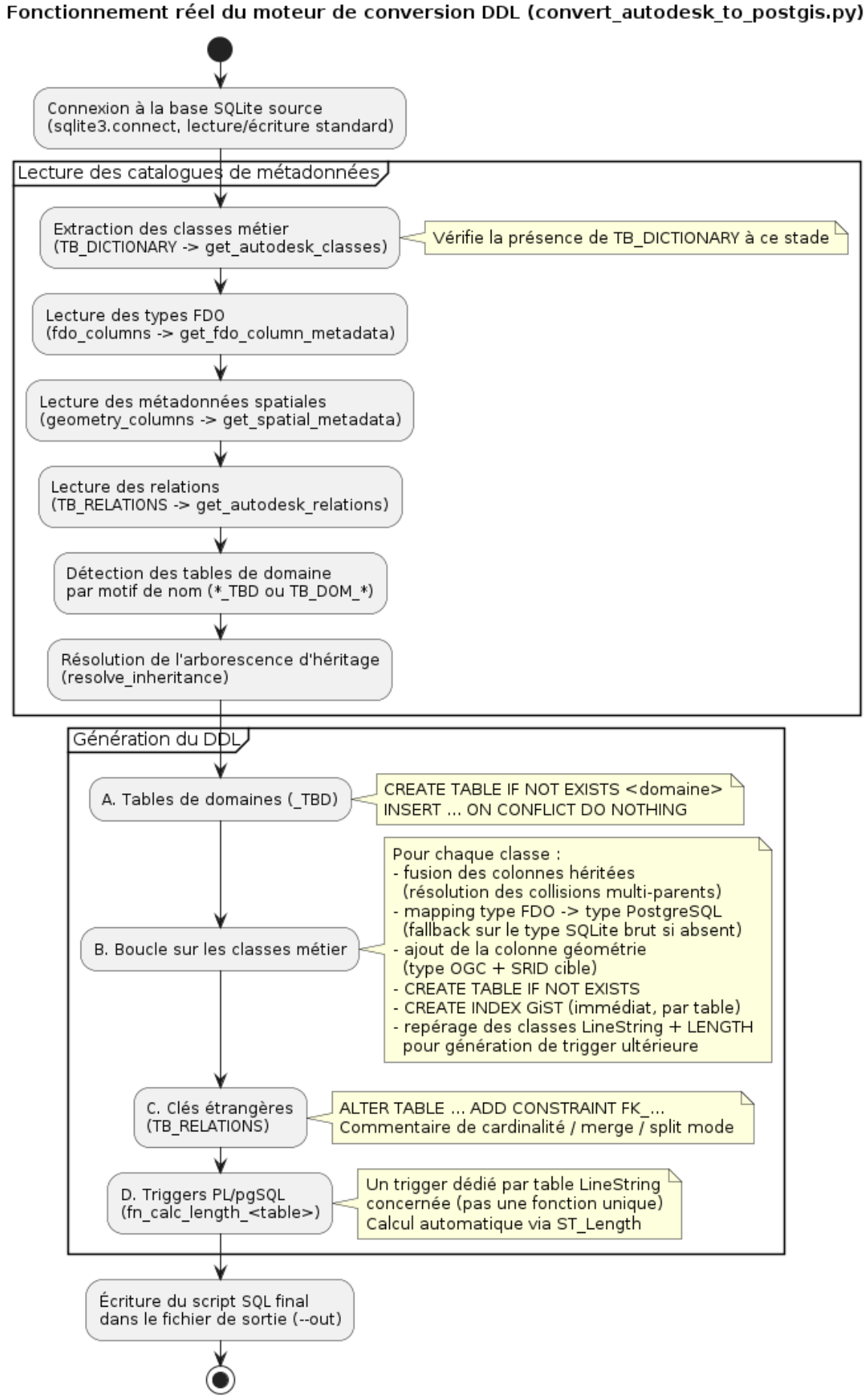


FIGURE 17 – Diagramme d'activité illustrant le fonctionnement séquentiel du moteur de conversion DDL

7.2 Moteur de surveillance et synchronisation temps réel

Pour rendre le processus totalement transparent pour l'utilisateur, le script `watch_and_sync.py` assure la surveillance continue du système de fichiers en s'appuyant sur la bibliothèque `watchdog`.

7.2.1 Algorithme de détection dynamique dans %TEMP%

Autodesk Infrastructure Administrator décompresse les bases SQLite temporaires dans des sous-dossiers aux identifiants GUID dynamiques sous `%TEMP%` (ex : `AppData\Local\Temp\Embedded\<GUID>\`).

- Le service interroge le système de fichiers via `tempfile.gettempdir()` sans aucun chemin codé en dur.
- Il effectue une recherche récursive et sélectionne automatiquement l'instance la plus récente selon son horodatage de modification (`st_mtime`).

7.2.2 Algorithme de validation à 5 niveaux (is_autodesk_sqlite)

Pour éviter de générer inutilement des bases de données PostgreSQL lors de l'ouverture de simples dessins AutoCAD ou de fichiers systèmes, un algorithme de validation stricte à 5 niveaux a été développé :

1. **Filtre d'extension** : Rejet immédiat des fichiers non-bases de données (`.dwg`, `.bak`, `.xml`, etc.).
2. **Exclusion des bases système Autodesk** : Rejet ciblé de toutes les bases de catalogues internes d'Autodesk contenant la chaîne `tbsys` (`tbsys.sqlite`, `embeddedtbsys`, `tbsys_*.sqlite`). Les modèles métier aux noms légitimes (ex : `SysAssainissement.sqlite`) sont conservés.
3. **Vérification de l'en-tête magique SQLite** : Validation des 16 premiers octets du fichier (`SQLite format 3\x00`).
4. **Vérification de la table maîtresse** : Validation de l'existence effective de la table système `TB_DICTIONARY`.
5. **Filtrage des copies SQLite liées aux dessins DWG (has_associated_dwg)** : Différenciation fonctionnelle évitant de traiter les copies temporaires générées par AutoCAD pour ses dessins. L'algorithme effectue un scan ciblé des dossiers utilisateur (`Documents`, `Desktop`, `Downloads`) et vérifie la correspondance exacte du nom de fichier (*exact stem matching*) pour rejeter les copies secondaires tout en acceptant les vrais modèles métier.

7.2.3 Synchronisation non destructive

Afin de préserver les données géospatiales déjà saisies dans PostgreSQL :

- Les tables sont créées via `CREATE TABLE IF NOT EXISTS`.
- Les énumérations sont insérées via `INSERT ... ON CONFLICT DO NOTHING`.
- Les évolutions de structure appliquent `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`.
- Le nom de la base PostgreSQL est automatiquement dérivé du modèle (ex : `industry_model_initial`).

7.2.4 Fonctionnement séquentiel du moteur de surveillance et de synchronisation

Le script maître `watch_and_sync.py` fonctionne sous la forme d'un service d'arrière-plan réactif suivant la séquence d'exécution ci-dessous :

1. **Chargement de la configuration** : lecture de `config.env` pour les identifiants PostgreSQL et le SRID cible.
2. **Scan initial des modèles actifs** : recherche des modèles Autodesk présents dans `%TEMP%` ou un répertoire explicite.
3. **Démarrage du mode de surveillance** : mode événementiel (`watchdog`) par défaut, ou mode dégradé par scrutation périodique en son absence.
4. **Passage du pipeline de validation à 5 niveaux** : filtrage des fichiers détectés (extension, exclusion système, signature SQLite, présence de `TB_DICTIONARY`, absence de liaison DWG).

Lorsqu'un fichier est validé comme un véritable modèle Autodesk, le service déclenche le traitement complet du modèle, détaillé ci-après :

5. **Déduction du nom de la base cible** : génération dynamique du nom de la base PostgreSQL.
6. **Appel in-process du moteur de conversion** : génération du DDL en mémoire et écriture du script SQL sur disque.
7. **Application incrémentale du DDL** : exécution instruction par instruction sur PostgreSQL, avec gestion individuelle des erreurs.
8. **Synchronisation des évolutions du modèle** : ajout des nouvelles colonnes, suppression des éléments retirés, et synchronisation des données par upsert si activée.
9. **Journalisation et rapport d'audit** : traçabilité horodatée de chaque cycle de synchronisation.

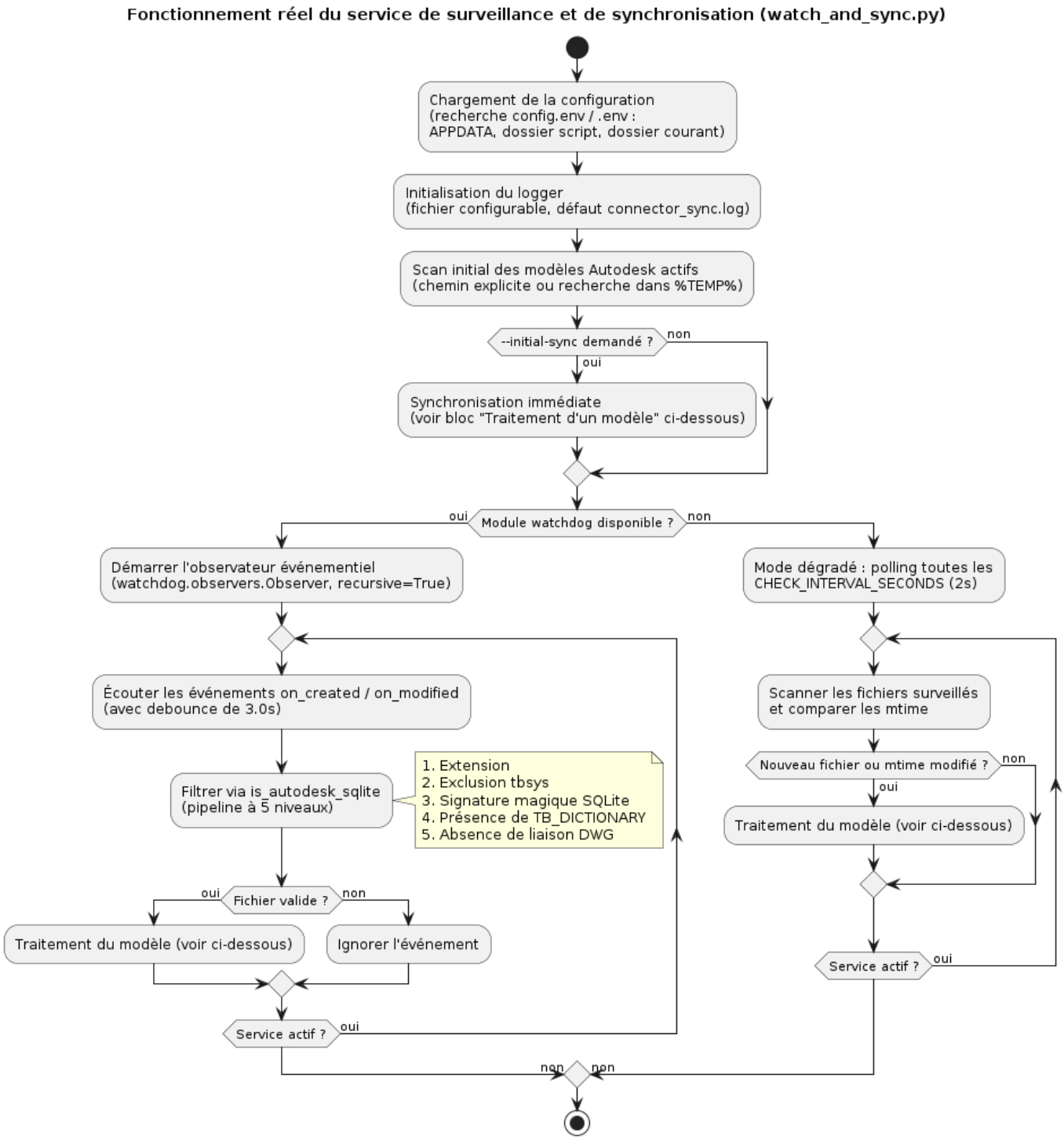


FIGURE 18 – Diagramme d'activité illustrant la boucle de surveillance du service `watch_and_sync.py`

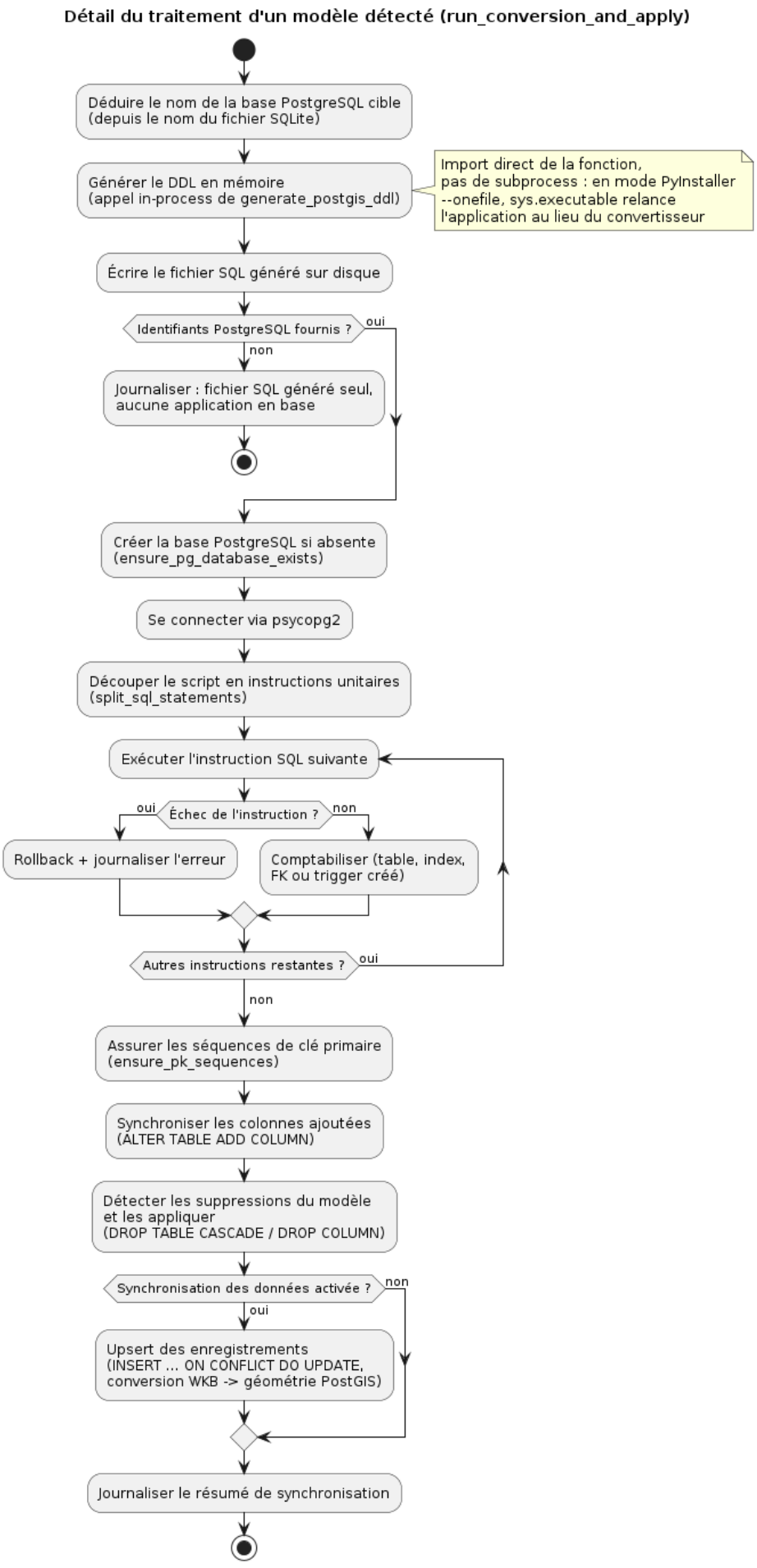


FIGURE 19 – Diagramme d'activité illustrant le traitement d'un modèle Autodesk détecté

7.3 Validation par tests automatisés (Suite Pytest)

Avant tout déploiement, le comportement des deux scripts a été rigoureusement validé par une suite de tests unitaires automatisés sous **Pytest** dans le répertoire `tests/` :

TABLE 6 – Suite de tests unitaires et d'intégration automatisés

Fichier de test	**Objectif et éléments contrôlés**
`tests/conftest.py`	Fixtures Pytest initialisant les bases de données SQLite temporaires et les connexions PostgreSQL de test.
`tests/test_converter.py`	Validation du moteur DDL : typage FDO, extraction des 6 catalogues maîtres, résolution des clés `FID`/`ID` et syntaxe SQL PostGIS.
`tests/test_watcher.py`	Validation de l'algorithme à 5 niveaux, filtrage des fichiers `tbsys`, détection des dossiers `%TEMP%` et persistance du service.
`tests/test_inheritance.py`	Validation du moteur d'héritage simple/multiple et résolution des collisions de colonnes.

Tous les tests unitaires ont été exécutés avec un taux de réussite de **100%**, confirmant la robustesse du code.

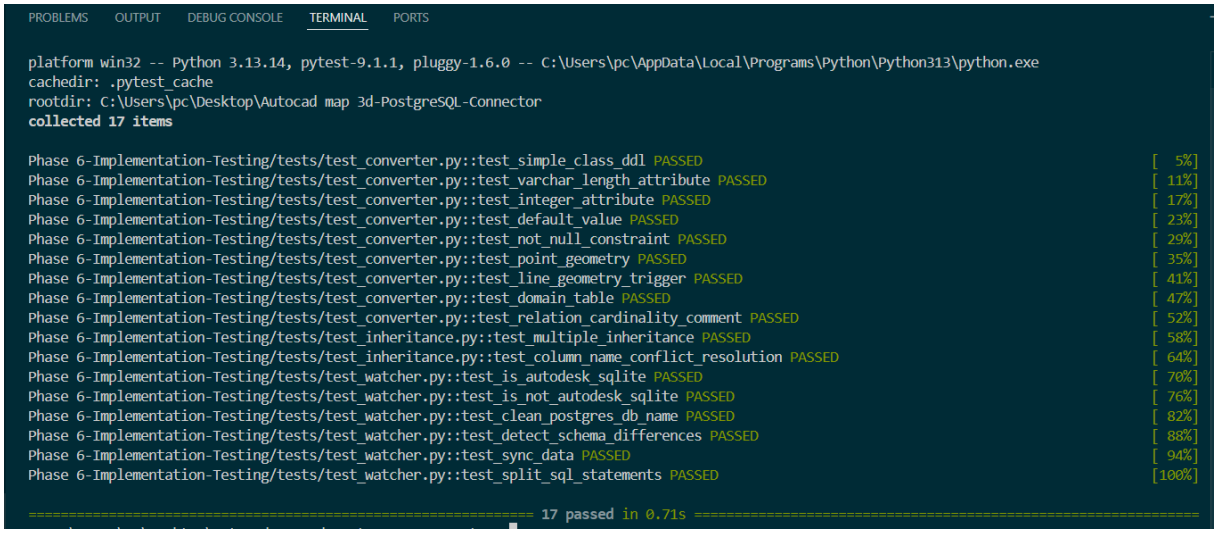


FIGURE 20 – Résultats des tests unitaires

7.4 Validation par tests manuels et commandes CLI

En complément des tests automatisés et avant le développement de l'interface graphique, la solution a été entièrement éprouvée en ligne de commande (CLI).

7.4.1 Commande de lancement manuelle

Le pipeline de synchronisation a d'abord été validé via la commande maître :

```
python watch_and_sync.py -pg-user postgres -pg-pass 123456 -initial-sync
```

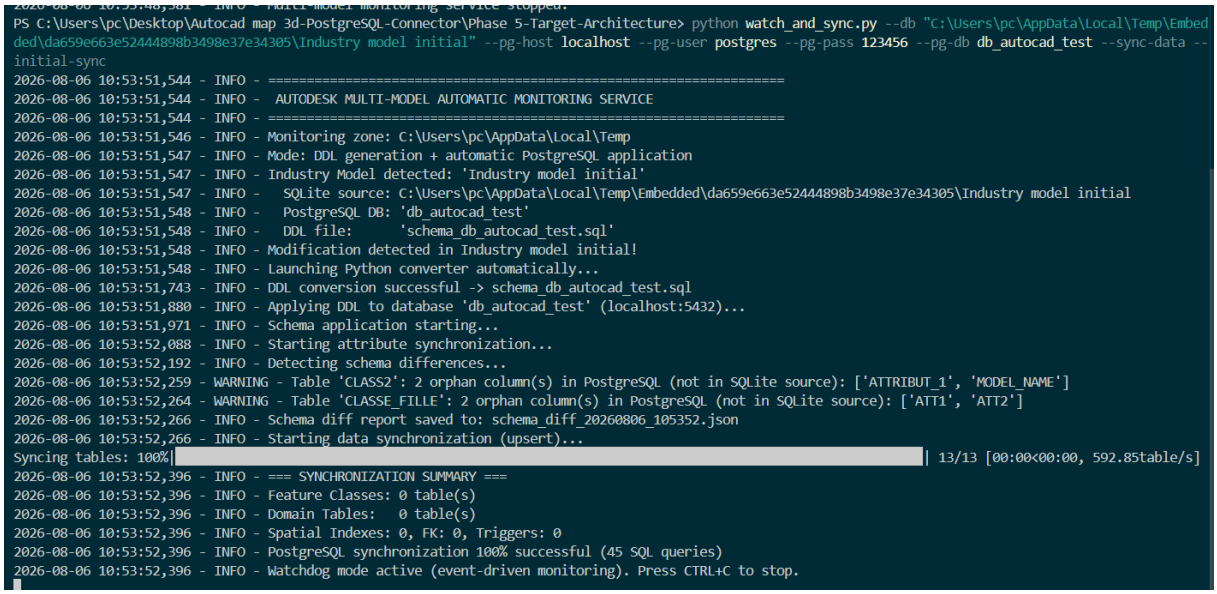


FIGURE 21 – Exécution et validation du pipeline de synchronisation en ligne de commande (CLI)

7.4.2 Scénarios de modification et de suppression physique

Les essais manuels ont permis de valider les scénarios fonctionnels suivants :

- **Ajout dynamique de classe/attribut** : La création d'une nouvelle classe (ex : *Vanne*) dans Infrastructure Administrator génère immédiatement la commande `CREATE TABLE` sous PostgreSQL. L'ajout d'un champ déclenche une commande `ALTER TABLE ... ADD COLUMN`.
- **Suppression physique cascade** : La suppression d'une classe ou d'un attribut est répercutée physiquement dans PostgreSQL via `DROP TABLE ... CASCADE` ou `ALTER TABLE ... DROP COLUMN`, accompagnée de la génération automatique d'un rapport d'audit JSON.

Ce bilan valide l'intégralité du moteur de synchronisation autonome en ligne de commande.

Chapitre 8: Application d'administration, packaging et déploiement Windows

Ce chapitre présente l'industrialisation de la solution pour les utilisateurs finaux. Il détaille le développement de l'application d'administration System Tray (gui_tray_app.py), les étapes de packaging autonome avec PyInstaller (build_exe.py), la génération de l'installateur Windows Inno Setup (Setup_PostMapLive.exe), ainsi que le guide d'utilisation sous AutoCAD Map 3D.

8.1 Interface d'administration System Tray (`gui_tray_app.py`)

Une application graphique résidant dans la zone de notification Windows (*System Tray*) a été développée en Python à l'aide des bibliothèques `pystray` et `Pillow`. Elle offre une visibilité directe et un contrôle complet sur le service d'arrière-plan sans encombrer l'espace de travail des cartographes.

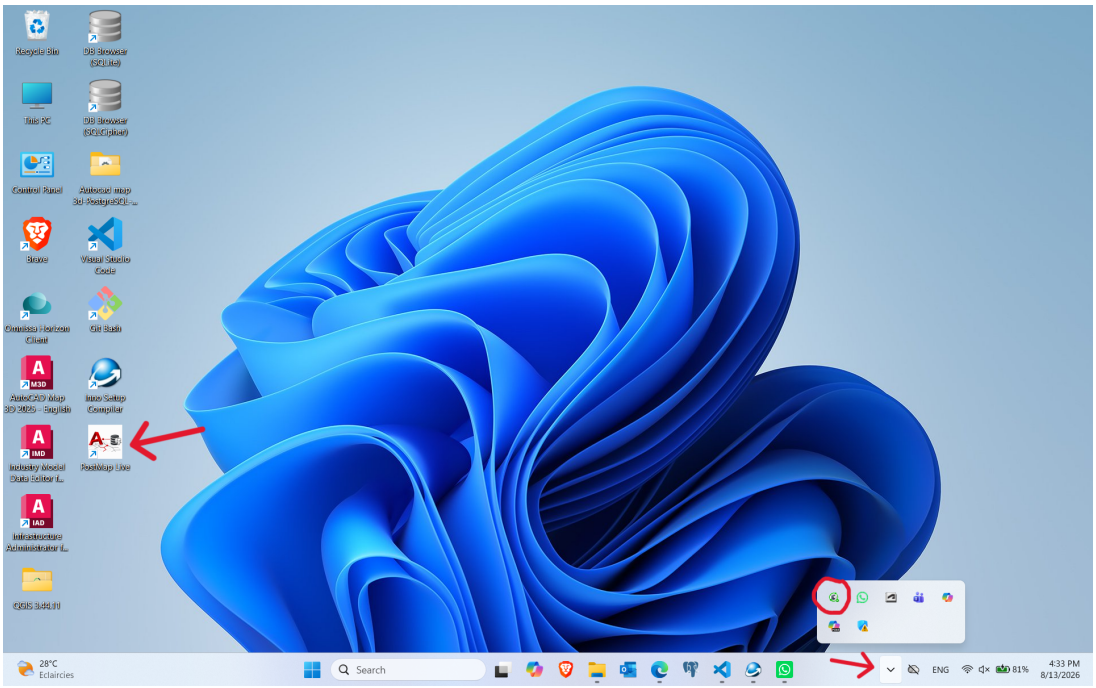


FIGURE 22 – Interface de l'application graphique d'administration System Tray

8.1.1 Gestion des états et icônes dynamiques

L'application indique visuellement l'état du service d'arrière-plan via des icônes colorées :

- **Actif (Active)** : Icône verte confirmant la surveillance active et la synchronisation automatique de la zone temporaire.
- **En pause (Paused)** : Icône orange indiquant la suspension temporaire du service.
- **Arrêté (Quit App)** : Service arrêté.



FIGURE 23 – État Actif (Vert)



FIGURE 24 – État En pause (Orange)

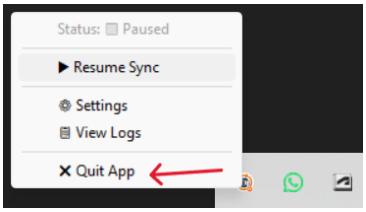


FIGURE 25 – Menu contextuel interactif

8.1.2 Fonctionnalités du menu contextuel

Le menu contextuel permet à l'utilisateur de démarrer/arrêter/mettre en pause le service, d'ouvrir l'emplacement de configuration `%APPDATA%\PostMapLive`, et de consulter les journaux de synchronisation en temps réel (`watch_and_sync.log`).

8.2 Procédure de packaging et compilation autonome (`build_exe.py`)

Pour permettre un déploiement fluide sur des postes de travail ne disposant pas d'un environnement Python ni de compilateur, l'application a été entièrement autonomisée à l'aide de **PyInstaller**. Le script d'automatisation de compilation `build_exe.py` a été développé pour regrouper l'interpréteur Python, les scripts métier et l'ensemble des bibliothèques tierces au sein d'un fichier exécutable binaire unique (`PostMapLive.exe`).

8.2.1 Fonctionnement et chaîne de compilation du script `build_exe.py`

Le script `build_exe.py` exécute une séquence ordonnée en 4 étapes :

1. **Résolution dynamique des chemins** : localisation des répertoires du projet.
2. **Construction de la commande PyInstaller** : assemblage des paramètres de compilation (`-onefile`, `-noconsole`, `-icon`, `-add-data`, `-hidden-import`).
3. **Nettoyage de l'exécutable précédent** : suppression de l'ancien binaire, ou interruption si le fichier est verrouillé.
4. **Exécution de PyInstaller et validation** : lancement du sous-processus et vérification du code de retour pour confirmer ou signaler l'échec.

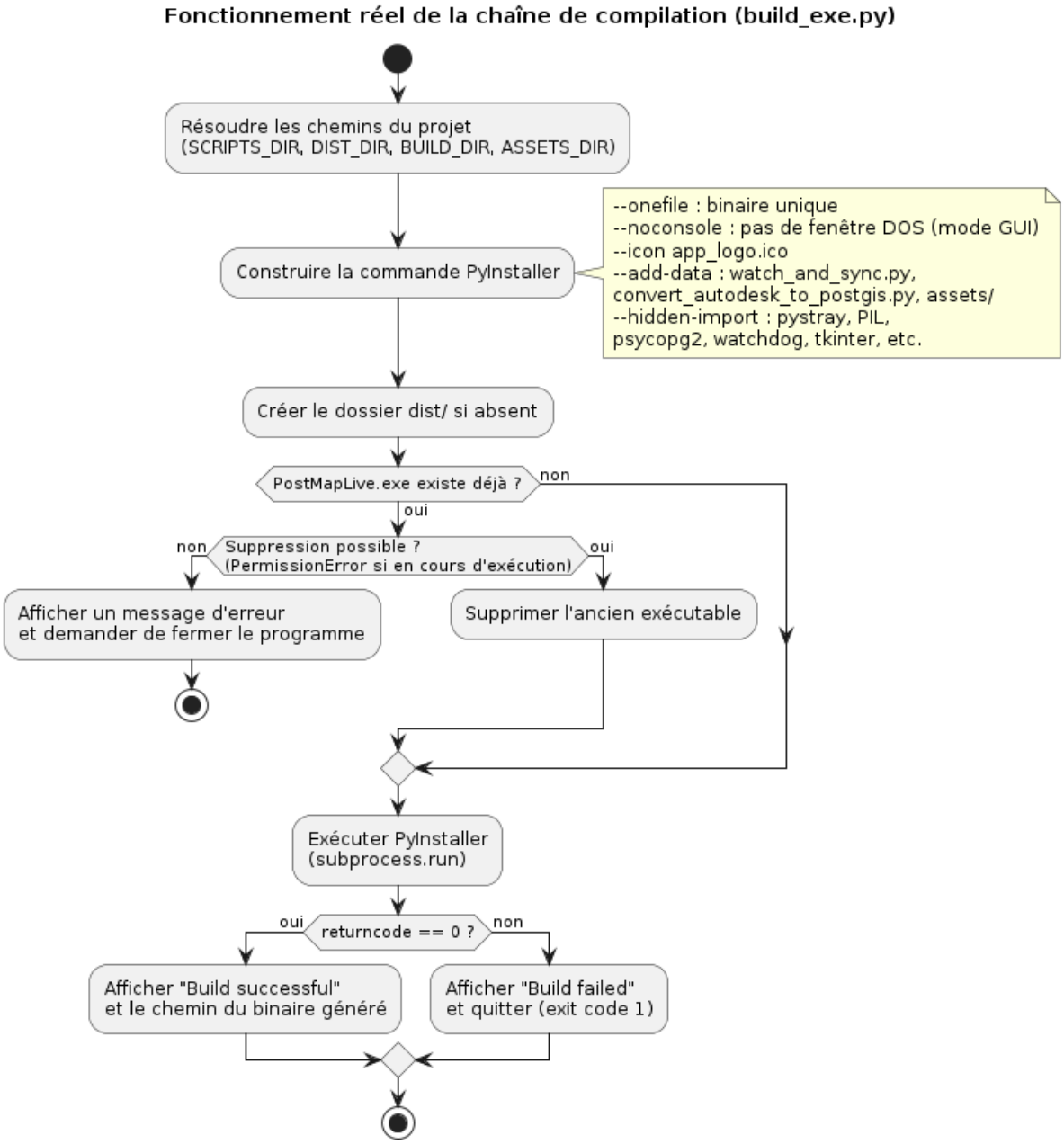


FIGURE 26 – Diagramme d'activité illustrant la chaîne de compilation du script `build_exe.py` en exécutable Windows autonome via PyInstaller

8.2.2 Gestion de l'arborescence temporaire runtime

Lors de l'exécution d'un binaire généré avec l'option `-onefile`, PyInstaller décompresse à la volée ses composants internes dans un répertoire temporaire système isolé nommé `_MEIXXXXXX` (accessible via `sys._MEIPASS`).

Afin d'éviter les erreurs d'accès de type `FileNotFoundError` qui survenaient lors d'appels externes par sous-processus (`subprocess`), le script d'entrée `gui_tray_app.py` et le moteur de synchronisation utilisent un système d'importation interne direct (*in-process*). Cette conception garantit que le moteur DDL et la surveillance s'exécutent entièrement en mémoire vive, sans dépendre de l'emplacement du répertoire courant d'exécution.

8.3 Création de l'installateur Windows Inno Setup

Afin d'assurer la distribution industrielle du logiciel sur les postes clients sans nécessiter de privilèges d'administration système avancés, un script de déploiement **Inno Setup** (`installer_setup.iss`) a été conçu. Ce script compile le binaire généré (`dist/PostMapLive.exe`) et crée l'assistant d'installation autonome `Setup_PostMapLive.exe`.

8.3.1 Structure et paramètres clés du script Inno Setup (`installer_setup.iss`)

Le script de déploiement s'articule autour des sections Inno Setup résumées dans le tableau ci dessous . L'installateur cible un déploiement sans droits administrateur (`PrivilegesRequired=lowest`) avec une compression solide `lzma2/ultra64` pour minimiser la taille du package.

TABLE 7 – Sections principales du script Inno Setup `installer_setup.iss`

Section	Rôle	Paramètres clés
`[Setup]`	Identité et comportement global de l'installateur	GUID unique (`AppId`), nom *PostMap Live*, version 1.0.0, répertoire par défaut `{autopf}\PostMap Live`, `PrivilegesRequired=lowest` (aucun droit admin requis)
`[Files]`	Fichiers embarqués dans le package	Copie de `dist\PostMapLive.exe` vers le répertoire d'installation `{app}`
`[Icons]` & `[Tasks]`	Raccourcis proposés à l'utilisateur	Raccourci menu Démarrer (systématique) et raccourci Bureau (optionnel, via case à cocher)
`[Registry]`	Démarrage automatique de l'application	Inscription dans `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` (optionnel, via case à cocher)
`[Run]`	Exécution post-installation	Lancement automatique de l'application System Tray dès la fermeture de l'assistant (`Flags: nowait postinstall`)
`[UninstallRun]` & `[UninstallDelete]`	Désinstallation propre	Fermeture forcée du processus (`taskkill /F /IM PostMapLive.exe`) puis suppression des fichiers de configuration et de journal dans le répertoire d'installation

8.3.2 Procédure de génération de l'installateur

La génération de l'installateur final suit un processus en 3 étapes simples :

1. Compilation de l'exécutable via le script Python : `python scripts/build_exe.py`.
2. Ouverture du fichier `installer_setup.iss` dans l'environnement Inno Setup Compiler.

3. Lancement de la compilation (*Build* → *Compile*), générant le fichier exécutable d'installation final dans `installer_output/Setup_PostMapLive.exe`.

Grâce à cette chaîne de déploiement automatisée, l'entreprise dispose d'un paquet d'installation autonome, léger et sécurisé, prêt à être déployé sur l'ensemble du parc informatique.

8.4 Synthèse du chapitre

Grâce au packaging sous forme d'installateur Windows autonome et à l'intégration discrète en barre des tâches, la solution s'insère parfaitement dans l'environnement de travail des cartographes, éliminant toute contrainte technique liée à la ligne de commande.

Chapitre 9: Guide d'installation et d'utilisation

Ce chapitre fournit le manuel d'utilisation et le guide d'installation pas à pas de la solution développée. Il détaille la procédure d'installation via l'assistant Windows (Setup_PostMapLive.exe), la configuration des identifiants PostgreSQL, ainsi que l'utilisation au quotidien du service en barre des tâches et de la connexion FDO sous AutoCAD Map 3D.

9.1 Guide d'installation pas à pas (`Setup_PostMapLive.exe`)

Pour simplifier le déploiement sur les postes de travail des cartographes et géomaticiens, l'ensemble des composants Python, dépendances et scripts d'arrière-plan ont été compilés sous la forme d'un installateur Windows autonome (`Setup_PostMapLive.exe`).

L'installation s'effectue en six étapes guidées :

1. **Lancement de l'assistant et choix du mode d'installation** : Exécution du fichier `Setup_PostMapLive.exe`. L'assistant propose de choisir entre une installation pour l'utilisateur courant uniquement (*Install for me only*, recommandé, ne nécessitant pas de droits administrateur) ou une installation pour tous les utilisateurs (*Install for all users*, nécessitant des privilèges administratifs).

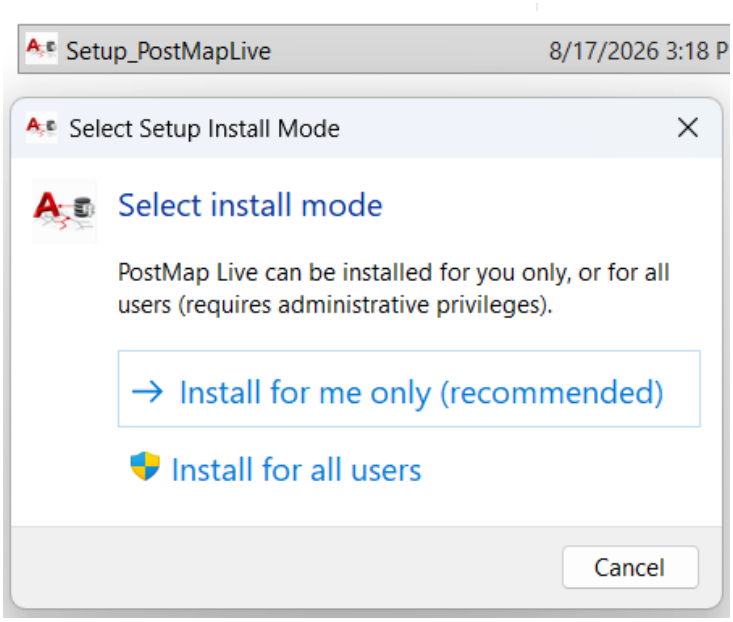


FIGURE 27 – Assistant d'installation — Étape d'accueil de Setup_PostMapLive.exe

2. **Sélection du répertoire de destination** : L'assistant propose par défaut le dossier `C:\Users\pc\AppData\Local\Programs\PostMap Live`, modifiable via le bouton *Browse...*. Un espace disque libre d'au moins 27.4 Mo est requis.

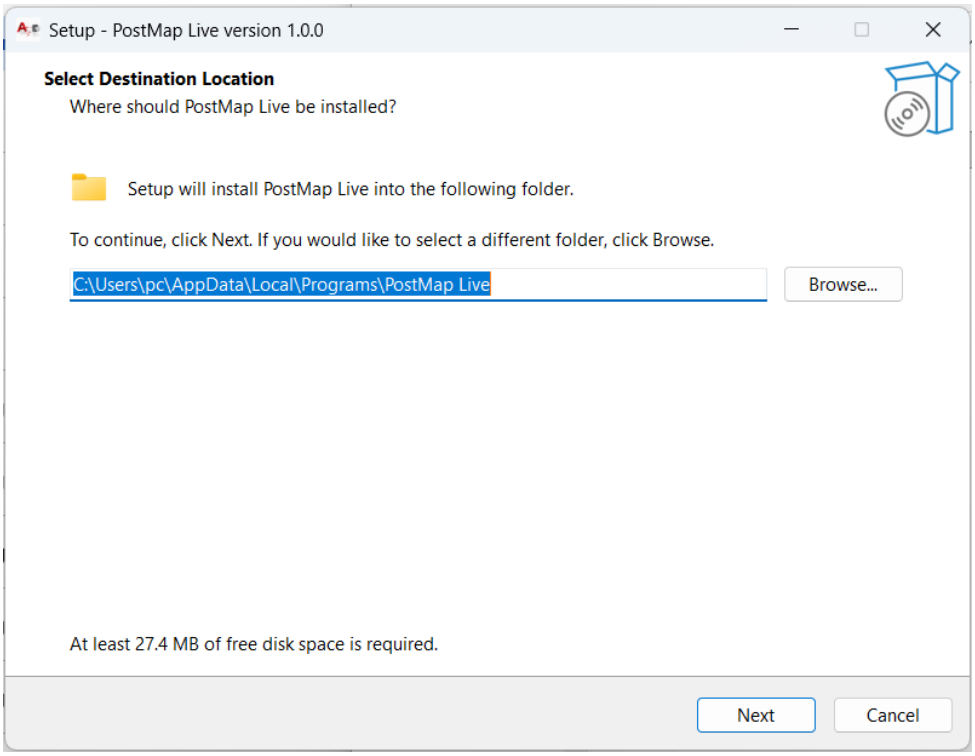


FIGURE 28 – Assistant d'installation — Choix du répertoire de destination

3. **Sélection du dossier du menu Démarrer** : L'assistant crée par défaut un dossier de raccourcis nommé `PostMap Live` dans le menu Démarrer de Windows, également modifiable.

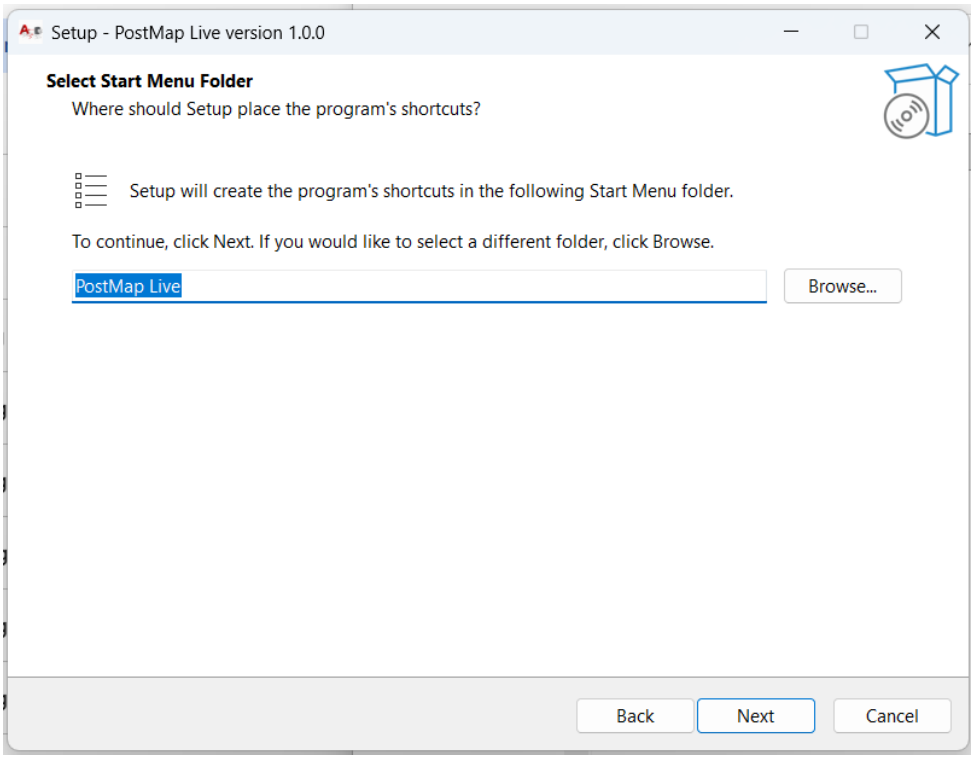


FIGURE 29 – Assistant d'installation — Sélection du dossier du menu Démarrer

4. **Sélection des tâches complémentaires** : L'utilisateur peut choisir de créer un raccourci sur le bureau (*Create a desktop shortcut*) et d'activer le lancement automatique de l'application au démarrage de Windows (*Launch automatically at Windows startup*).

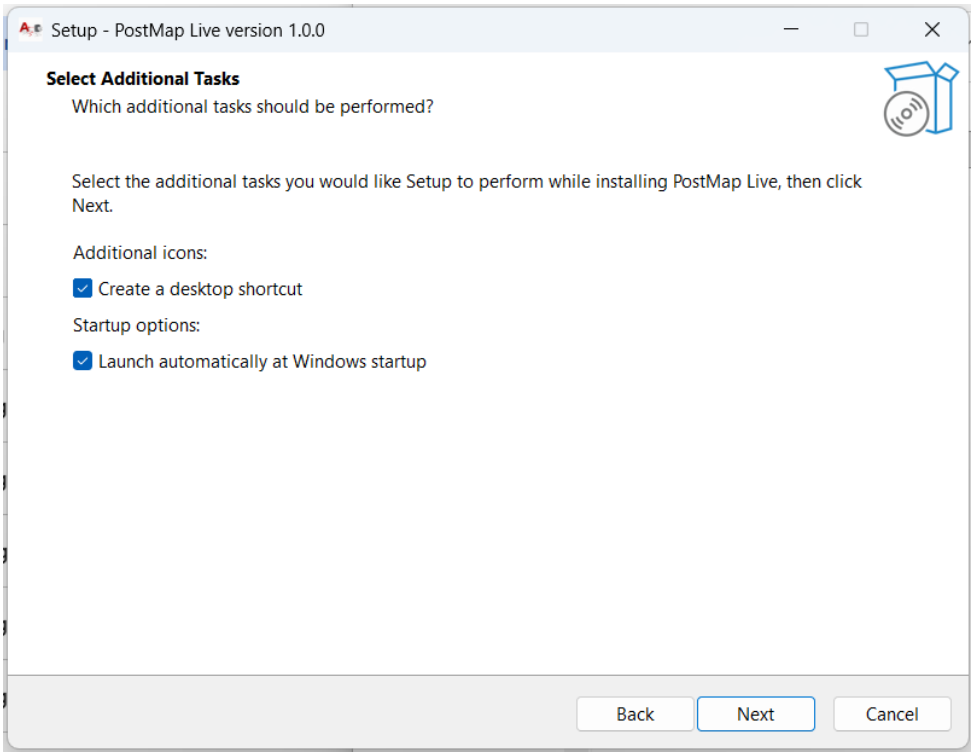


FIGURE 30 – Assistant d'installation — Sélection des tâches complémentaires

5. **Récapitulatif et confirmation** : Un écran récapitulatif affiche l'ensemble des paramètres sélectionnés (répertoire de destination, dossier du menu Démarrer, tâches complémentaires) avant de lancer l'installation proprement dite via le bouton *Install*.

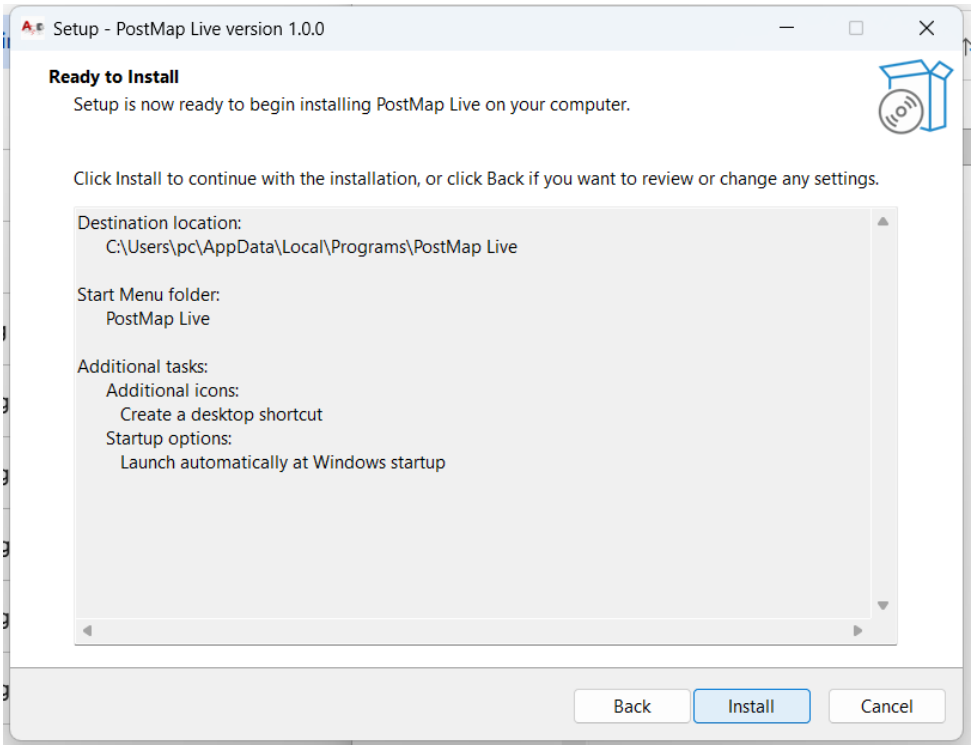


FIGURE 31 – Assistant d'installation — Récapitulatif avant installation

6. **Finalisation de l'installation** : Une fois la copie des fichiers terminée, l'assistant confirme la réussite de l'installation et propose de lancer immédiatement l'application via la case *Launch PostMap Live now*.

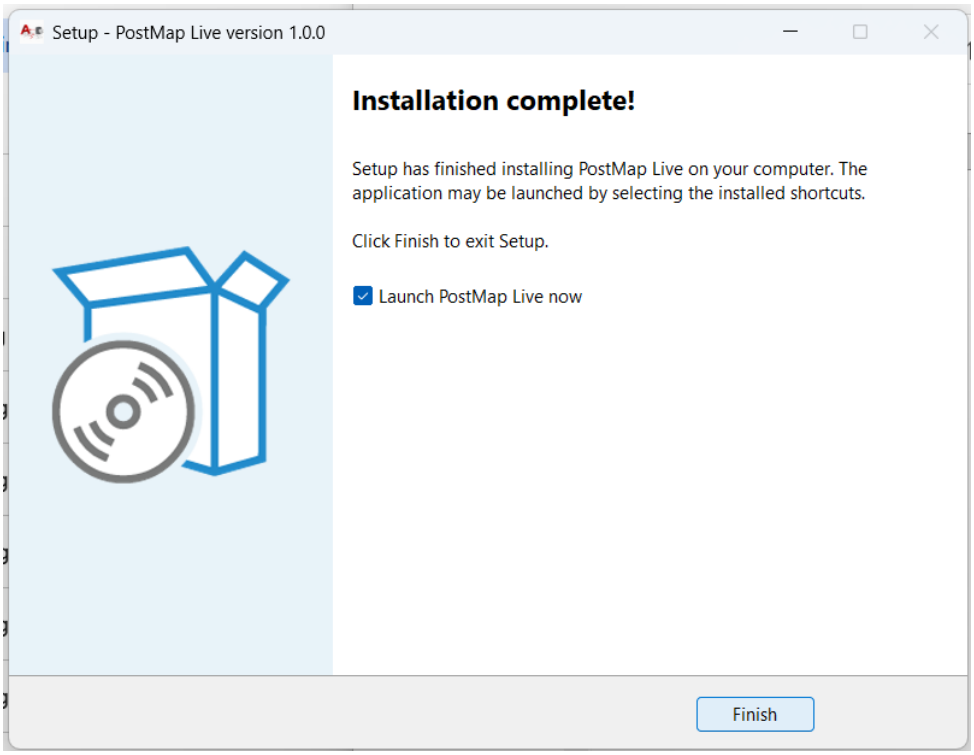


FIGURE 32 – Assistant d'installation — Finalisation et lancement du service

9.2 Guide d'utilisation au quotidien

Le fonctionnement de la solution est entièrement automatisé et réside dans la zone de notification Windows (*System Tray*).

9.2.1 Localisation et accès à l'application

Une fois installée, l'application *PostMap Live* se lance automatiquement et son icône apparaît dans la zone de notification Windows, à proximité de l'horloge système.

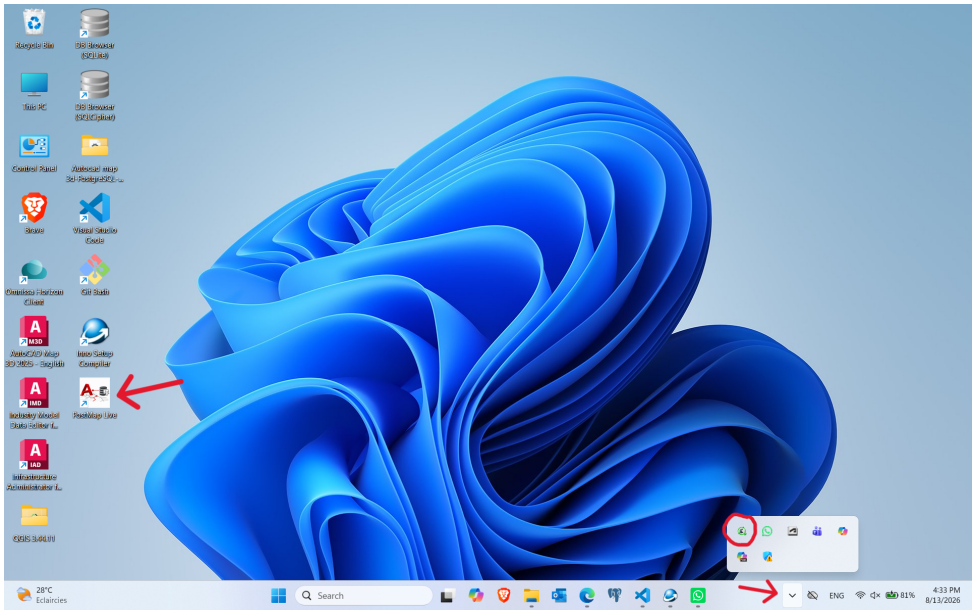


FIGURE 33 – Localisation de l'icône System Tray de PostMap Live dans la barre des tâches Windows

9.2.2 Menu contextuel et suivi de l'état du service

Un clic droit sur l'icône ouvre le menu contextuel, qui affiche en temps réel le statut du service (*Active* ou *Paused*) et permet de basculer manuellement la synchronisation via les options *Pause Sync* / *Resume Sync*.

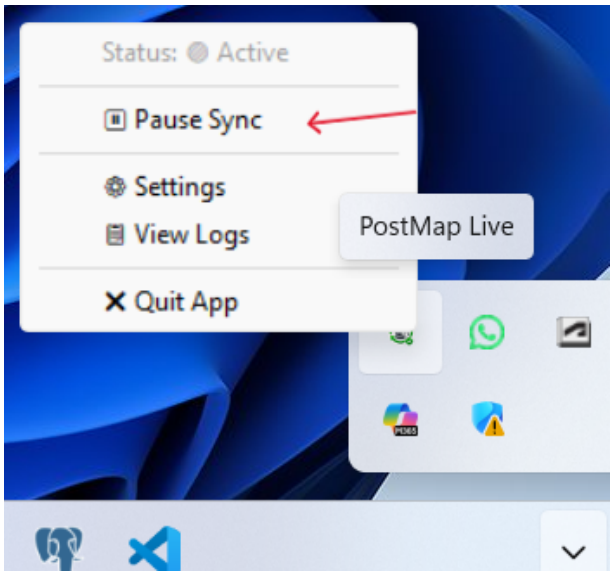


FIGURE 34 – Menu contextuel — Service actif, option de mise en pause (*Pause Sync*)

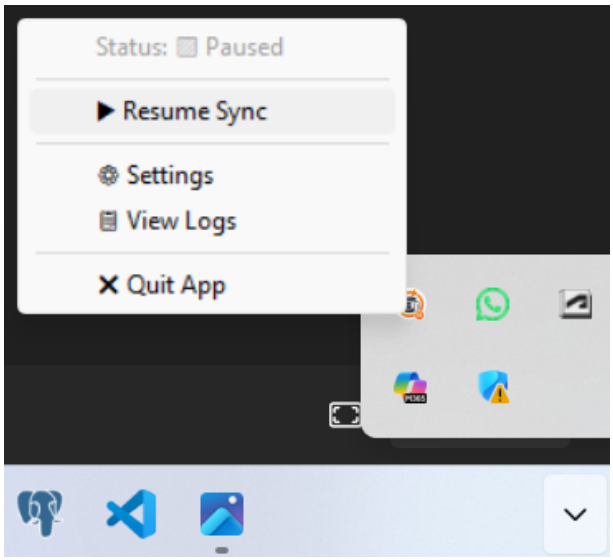


FIGURE 35 – Menu contextuel — Service en pause, option de reprise (*Resume Sync*)

9.2.3 Configuration initiale des accès PostgreSQL

Lors du premier lancement de l'application, l'administrateur ou l'utilisateur accède à la fenêtre de configuration via l'option *Settings* du menu contextuel, puis renseigne les paramètres de connexion au serveur PostgreSQL hôte (hôte, port, utilisateur, mot de passe) ainsi que le code SRID cible.

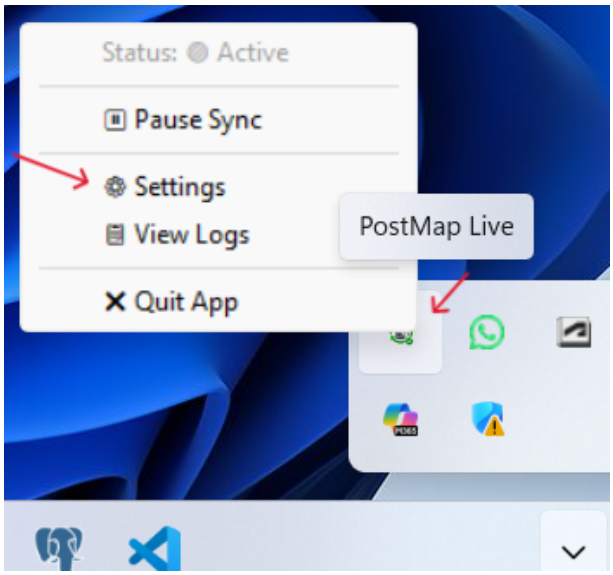


FIGURE 36 – Accès à la fenêtre de configuration via le menu contextuel (*Settings*)

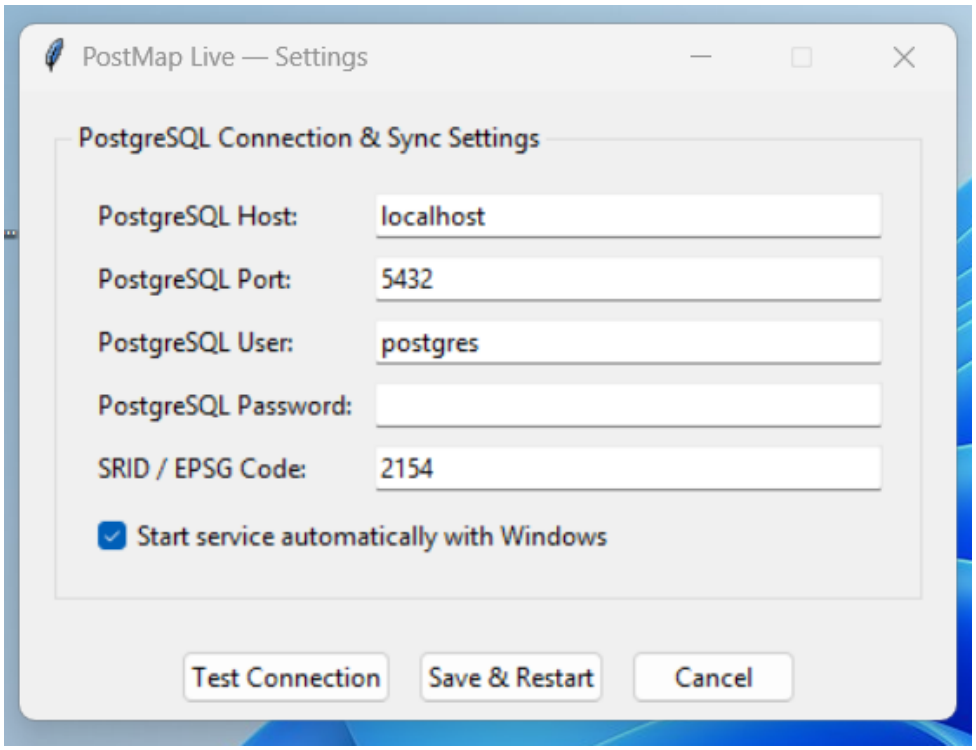


FIGURE 37 – Fenêtre de configuration PostMap Live — Paramètres de connexion PostgreSQL et code SRID

Ces paramètres sont sauvegardés de manière sécurisée dans le fichier de configuration `%APPDATA%\PostMapLive\config.env`. Une fois cette configuration enregistrée et validée (bouton *Test Connection*), l'utilisateur n'a plus jamais besoin d'ouvrir de terminal.

9.2.4 Consultation des journaux de synchronisation

L'option *View Logs* du menu contextuel ouvre une fenêtre affichant en temps réel le déroulement des cycles de synchronisation : détection du modèle Autodesk, génération

du DDL, création des tables de domaines et des classes métier, des index spatiaux et des triggers, puis résumé de la synchronisation.

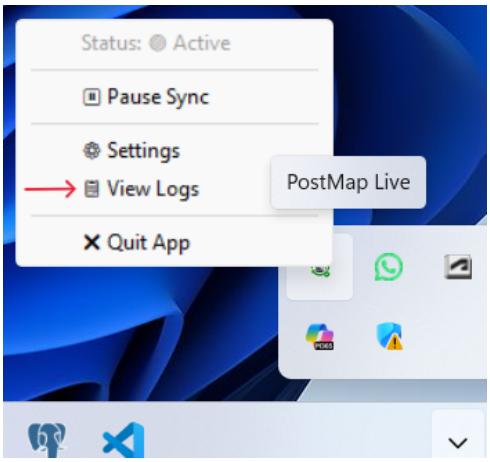


FIGURE 38 – Accès à la fenêtre des journaux via le menu contextuel (*View Logs*)

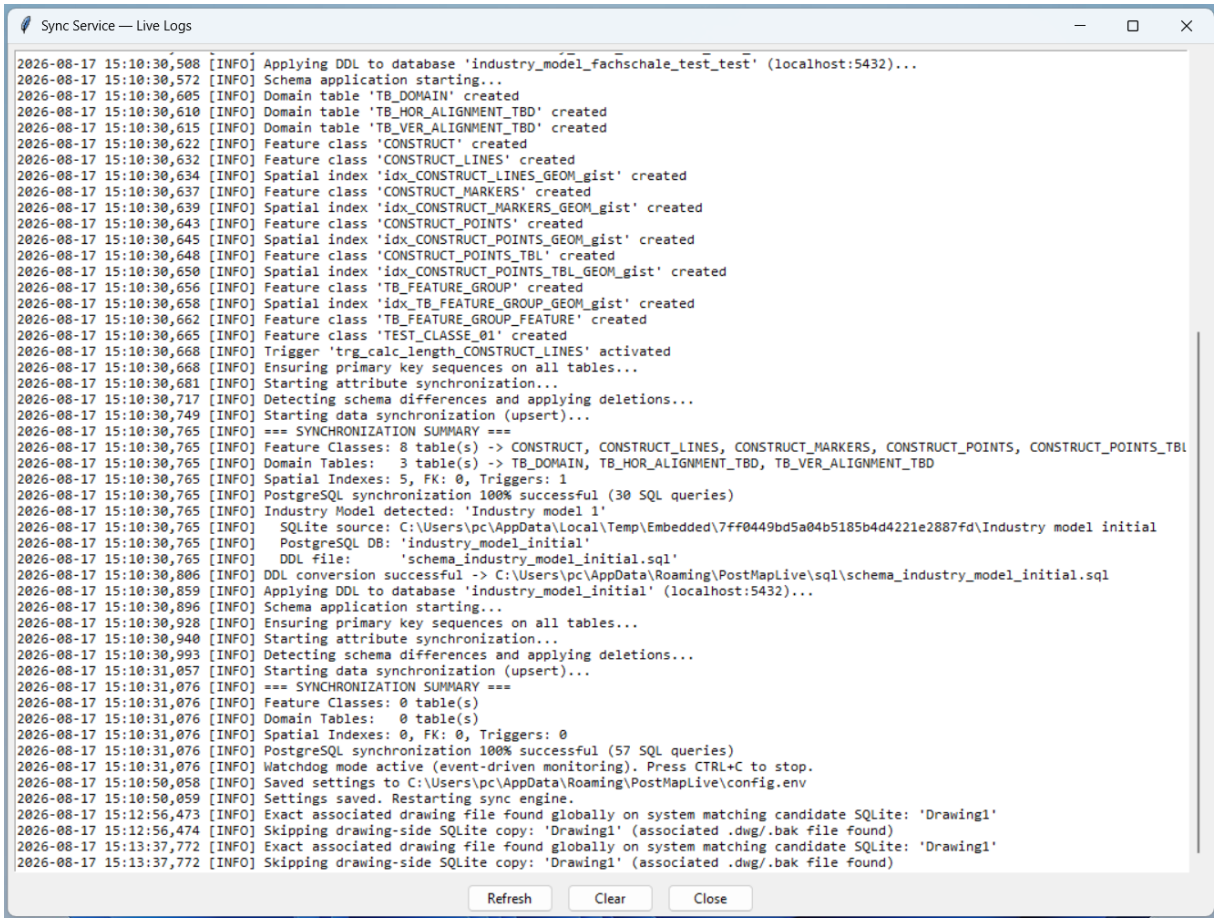


FIGURE 39 – Fenêtre des journaux en temps réel — Détection du modèle, génération et application du schéma PostGIS

9.2.5 Workflow de synchronisation automatique et édition sous AutoCAD Map 3D

Le workflow de travail pour le cartographe se déroule en trois étapes simples :

1. **Création/Ouverture du modèle sous Infrastructure Administrator** : L'utilisateur ouvre ou modifie un modèle d'industrie. Le service d'arrière-plan détecte automatiquement l'instance SQLite dans `%TEMP%`, génère le schéma SQL et crée la base PostgreSQL correspondante sous le nom du modèle.
2. **Connexion FDO dans AutoCAD Map 3D** : Dans AutoCAD Map 3D, l'utilisateur ouvre le volet *Data Connect* (→ *Add PostgreSQL Connection*), indique les identifiants et sélectionne la base générée.
3. **Édition cartographique temps réel** : Les couches métier apparaissent dans le *Display Manager*. Toute modification, ajout ou suppression de géométrie effectuée par le cartographe est directement enregistrée dans la base PostgreSQL.

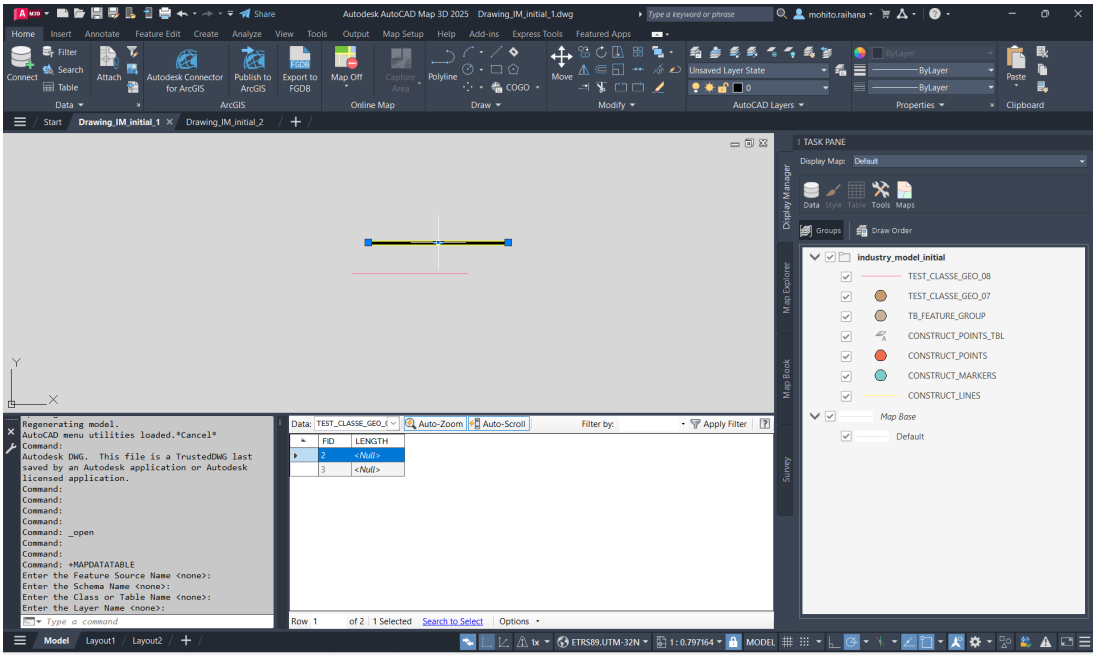


FIGURE 40 – Classes métier synchronisées affichées dans AutoCAD Map 3D (`Drawing_IM_initial_1`)

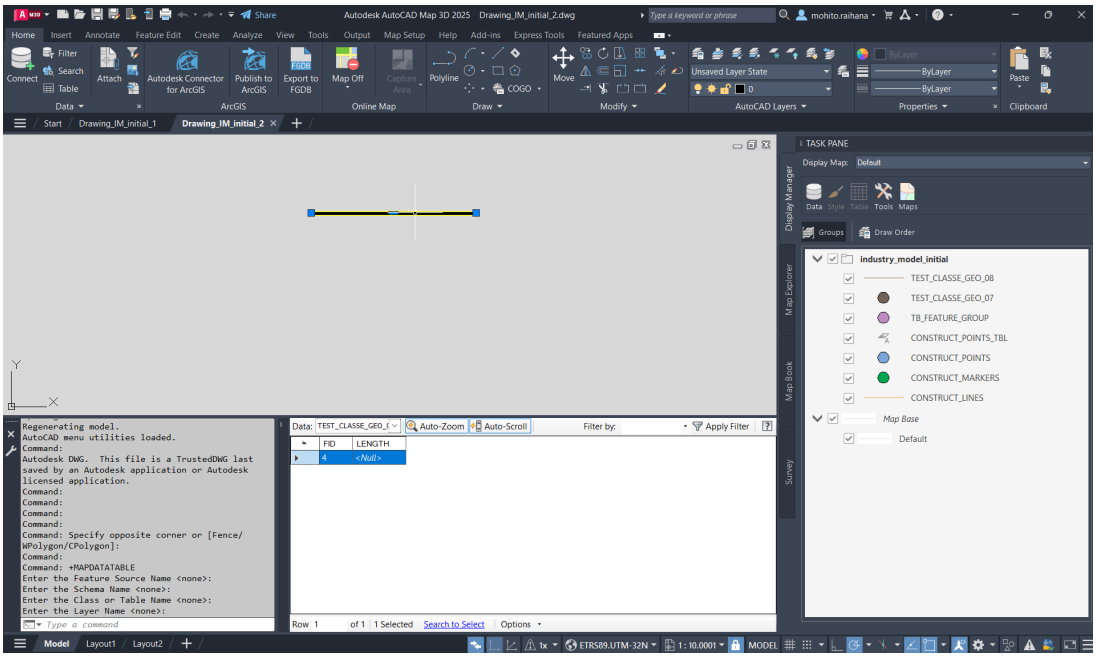


FIGURE 41 – Classes métier synchronisées affichées dans AutoCAD Map 3D (`Drawing_IM_initial_2`)

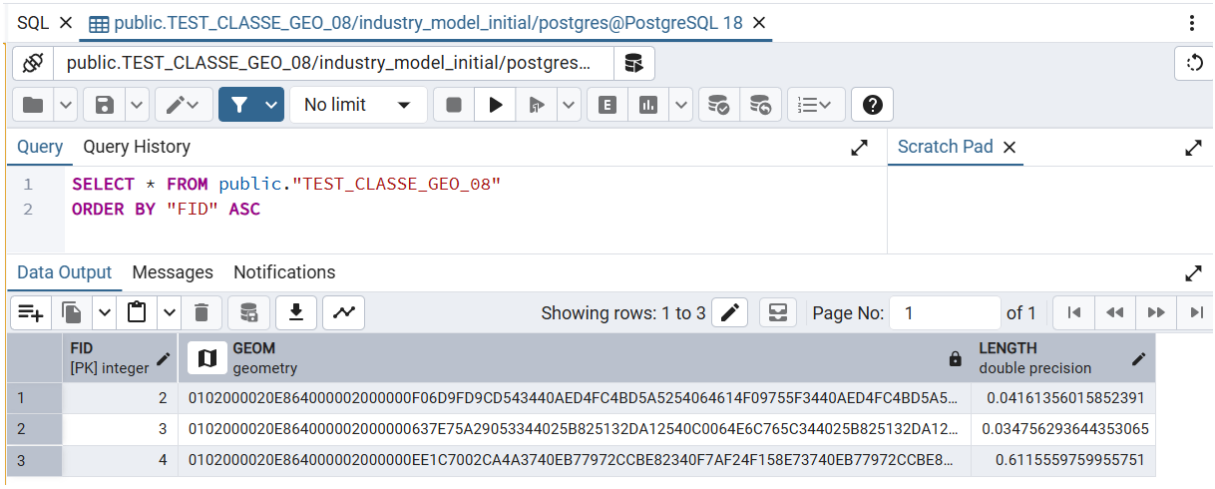


FIGURE 42 – Vérification des données synchronisées sous pgAdmin — table `TEST_CLASSE_GEO_08` avec géométrie et longueur calculée automatiquement

9.2.6 Indépendance des dessins et isolation multi-fichiers DWG

Un avantage fondamental du connecteur FDO natif réside dans l'isolation du contexte de travail entre plusieurs fichiers dessins (`.dwg`). Même lorsque plusieurs dessins AutoCAD Map 3D sont simultanément connectés à la même base de données PostgreSQL/PostGIS centralisée, chaque fichier `.dwg` conserve son propre environnement d'affichage dans le *Display Manager*.

Les requêtes spatiales, les filtres d'affichage et les sélections de couches configurés dans un dessin n'affectent en rien le contenu visualisé dans un autre dessin (comme illustré sur les figures de `Drawing_IM_initial_1` et `Drawing_IM_initial_2`). Les données géospatiales

sont centralisées et cohérentes dans PostgreSQL, mais l'expérience cartographique de chaque utilisateur reste parfaitement cloisonnée au sein de son propre fichier dessin.

9.3 Organisation des fichiers installés

TABLE 8 – Arborescence et rôle des fichiers installés sur le poste client

Fichier / Dossier	**Description / Rôle**
`C:\Users\<Nom_Utilisateur>\AppData\Local\Programs\PostMapLive`	Exécutable autonome principal (System Tray + Moteur de synchro)
`%APPDATA%\PostMapLive\config.env`	Fichier de configuration des identifiants et du SRID
`%APPDATA%\PostMapLive\watch_and_sync.log`	Journal horodaté des événements de détection et synchronisation

9.4 Synthèse du guide

Grâce au packaging sous forme d'installateur Windows et à l'intégration transparente dans la barre des tâches, la solution ne nécessite aucune connaissance technique ou ligne de commande de la part des cartographes. L'installation s'effectue en quelques clics et la synchronisation entre Autodesk et PostgreSQL s'exécute de manière totalement transparente en arrière-plan.

Chapitre 10: Conteneurisation et observabilité serveur

Ce chapitre présente la mise en place de l'infrastructure serveur conteneurisée. Il détaille l'organisation des services sous Docker Compose, la collecte automatisée de métriques via Prometheus, la visualisation synthétique sous Grafana, ainsi que les procédures de journalisation et de sauvegarde continue des données spatiales.

10.1 Motivation et objectifs d'observabilité

Afin d'assurer la haute disponibilité de la base de données PostgreSQL/PostGIS et d'offrir aux administrateurs réseau/SIG une visibilité complète sur la santé de l'infrastructure, la couche serveur a été entièrement isolée et conteneurisée via **Docker**.

Cette approche répond à trois objectifs stratégiques :

1. **Portabilité et déploiement en une commande** : Déploiement instantané d'une instance PostGIS et de sa stack de supervision sans installation manuelle complexe sur la machine hôte.
2. **Isolation et sécurité réseau** : Confinement des conteneurs au sein d'un réseau bridge virtuel étanche.
3. **Supervision temps réel (Observabilité)** : Collecte continue des métriques de performance et des transactions SQL pour anticiper les pannes et surveiller la charge de synchronisation.

10.2 Architecture conteneurisée unifiée (`docker-compose.yml`)

L'ensemble de l'infrastructure serveur est décrite au sein du fichier `docker-compose.yml` localisé dans le dossier `docker/` du projet, sous le nom de projet unifié `postmap-live`.

TABLE 9 – Architecture des conteneurs de la stack d'observabilité

Service Docker	Image / Port	Rôle fonctionnel
`postgres`	`postgis/postgis:15-3.3` (5432)	SGBD spatial PostgreSQL 15 avec l'extension PostGIS 3.3, hébergeant les modèles métier synchronisés.
`postgres_exporter`	`prometheuscommunity/postgres-exporter` (9187)	Agent d'introspection lisant les statistiques de requêtes, connexions et transactions de la base.
`prometheus`	`prom/prometheus:latest` (9090)	Serveur de collecte d'horodatage et de stockage des séries temporelles via `prometheus.yml`.
`grafana`	`grafana/grafana:latest` (3000)	Serveur de rendu visuel fournissant les tableaux de bord de supervision de l'infrastructure.

Le lancement de l'ensemble des 4 services s'effectue au moyen de la commande d'admi-

nistration :

```
cd docker
docker compose up -d
```

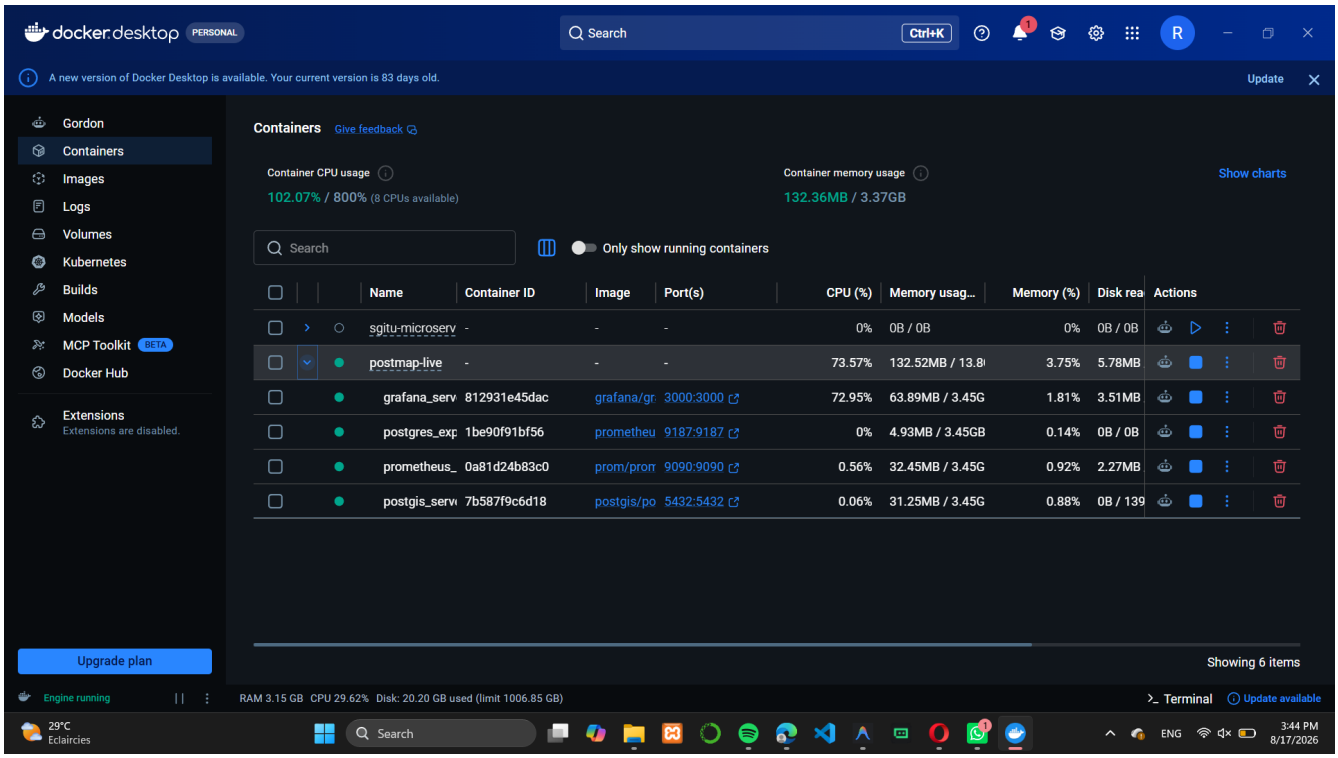


FIGURE 43 – Capture d'écran des services lancés

10.3 Mécanisme de collecte et configuration Prometheus (`prometheus.yml`)

Le serveur Prometheus agit comme le moteur de collecte de métriques. Sa configuration (`prometheus.yml`) définit un intervalle de réévaluation et de collecte (*scrape interval*) fixé à 15 secondes.

L'agent `postgres_exporter` extrait périodiquement les vues système de PostgreSQL (`pg_stat_database`, `pg_stat_activity`) et les transforme au format standard d'exposition Prometheus sur le port `9187`.

10.4 Supervision visuelle et tableaux de bord Grafana

L'interface web Grafana (`http://localhost:3000`) permet aux administrateurs de visualiser l'état de la base PostgreSQL grâce à l'importation du tableau de bord officiel PostgreSQL (ID `9628`).

Le tableau de bord permet de surveiller en temps réel :

— **Le débit de transactions par seconde** : Mesure des commits et rollbacks via la requête PromQL `rate(pg_stat_database_xact_commit_total[5m])`.

— **Le nombre de connexions actives** : Suivi du nombre d'éditeurs cartographiques simultanés sous AutoCAD Map 3D.

— **L'évolution du volume de stockage** : Taille physique de chaque base de données métier générée.

— **L'état du service de collecte** : Indicateur de statut *Target Status UP* confirmant la santé de la base.

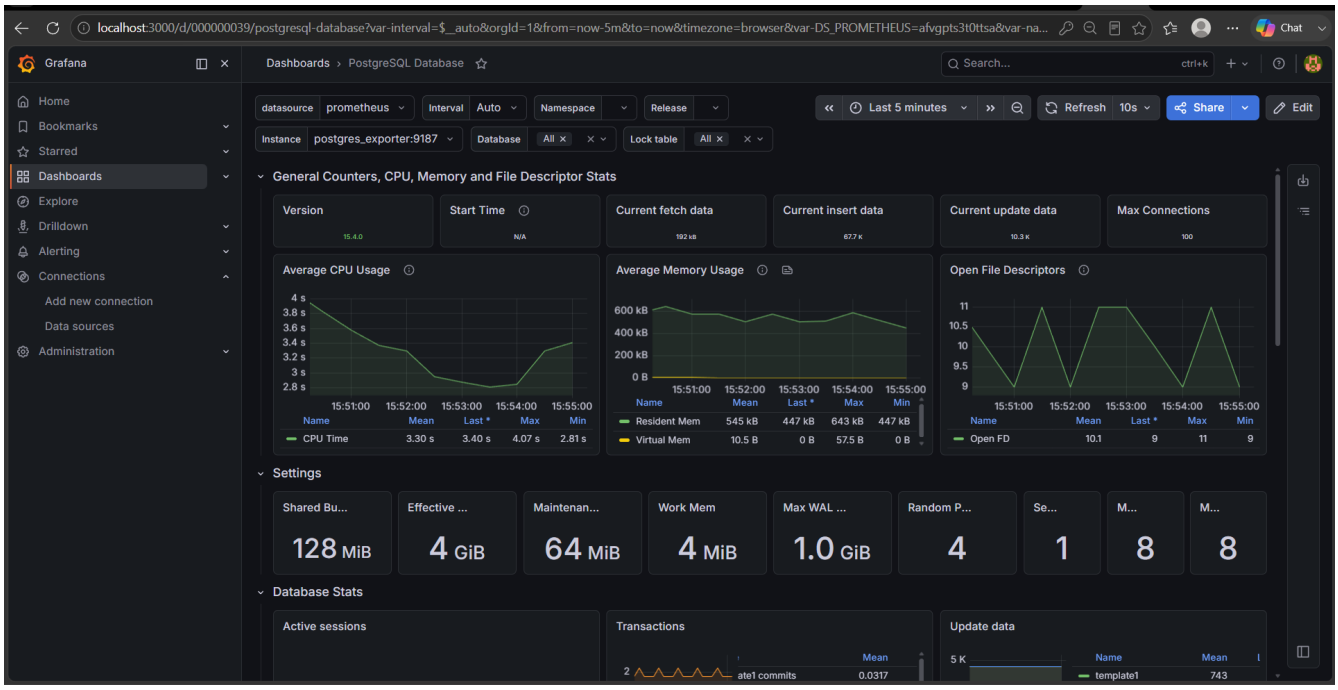


FIGURE 44 – Capture d'ecran de l'interface web Grafana (http ://localhost :3000)

10.5 Journalisation, persistance et sauvegardes (`pg_dump`)

La pérennité de l'infrastructure est garantie à deux niveaux :

1. **Persistance par volumes Docker** : Trois volumes managés (`pgdata`, `promdata`, `grafanadata`) sont rattachés aux conteneurs, préservant l'intégralité des données géospatiales et des métriques en cas de redémarrage ou de mise à jour du serveur.
2. **Stratégie de sauvegarde continue** : Un script automatisé d'export de dumps SQL via l'outil natif `pg_dump` permet de réaliser des sauvegardes à chaud récurrentes de la base conteneurisée, assurant la sécurité des données métiers.
3. **Journalisation centralisée** : Les logs du service client sont centralisés dans `%APPDATA%\PostMapL` tandis que les logs système du serveur PostgreSQL restent accessibles via la commande `docker logs postgis_server`.

10.6 Synthèse et bilan d'infrastructure

La mise en place de cette stack conteneurisée unifiée (PostGIS, Prometheus, Grafana) transforme le connecteur open-source en une véritable solution d'entreprise. Elle garantit aux responsables SIG et aux administrateurs IT une fiabilité opérationnelle, une isolation logicielle complète et une visibilité temps réel sur les performances du système.

Chapitre 11: Gestion du projet et collaboration

Ce chapitre présente l'organisation du travail tout au long du stage : la méthode de gestion de projet adoptée, le suivi des tâches, les outils de communication utilisés au sein de l'équipe, ainsi que la gestion des versions du code source.

11.1 Méthodologie de gestion de projet

Le projet a été mené selon une organisation itérative inspirée des méthodes agiles, adaptée à la durée et au contexte d'un stage de fin d'année. Le travail a été découpé en phases successives, correspondant globalement aux chapitres du présent rapport : découverte de l'architecture Autodesk, validation de la connectivité PostgreSQL/PostGIS natif, rétro-ingénierie du Data Model SQLite, analyse fonctionnelle de TKI PGP, conception de l'architecture cible, développement et packaging de l'application cliente Windows, rédaction des guides d'utilisation et de test, puis déploiement de la stack d'observabilité conteneurisée sous Docker. Chaque phase faisait l'objet d'un point d'avancement régulier avec les encadrants professionnels, permettant d'ajuster les priorités et de valider les résultats intermédiaires avant de passer à l'étape suivante.

11.2 Suivi des tâches

Le suivi des tâches a été organisé à l'aide de l'outil de gestion **Jira**, permettant de décomposer chaque phase en tickets individuels (tests de rétro-ingénierie, développement de fonctions du moteur de conversion, rédaction de la documentation, etc.). Cette décomposition a facilité la priorisation du travail au quotidien et a fourni une trace exploitable de l'avancement du projet, utile notamment pour la rédaction du présent rapport et pour objectiver la charge de travail réalisée sur chaque axe du projet.

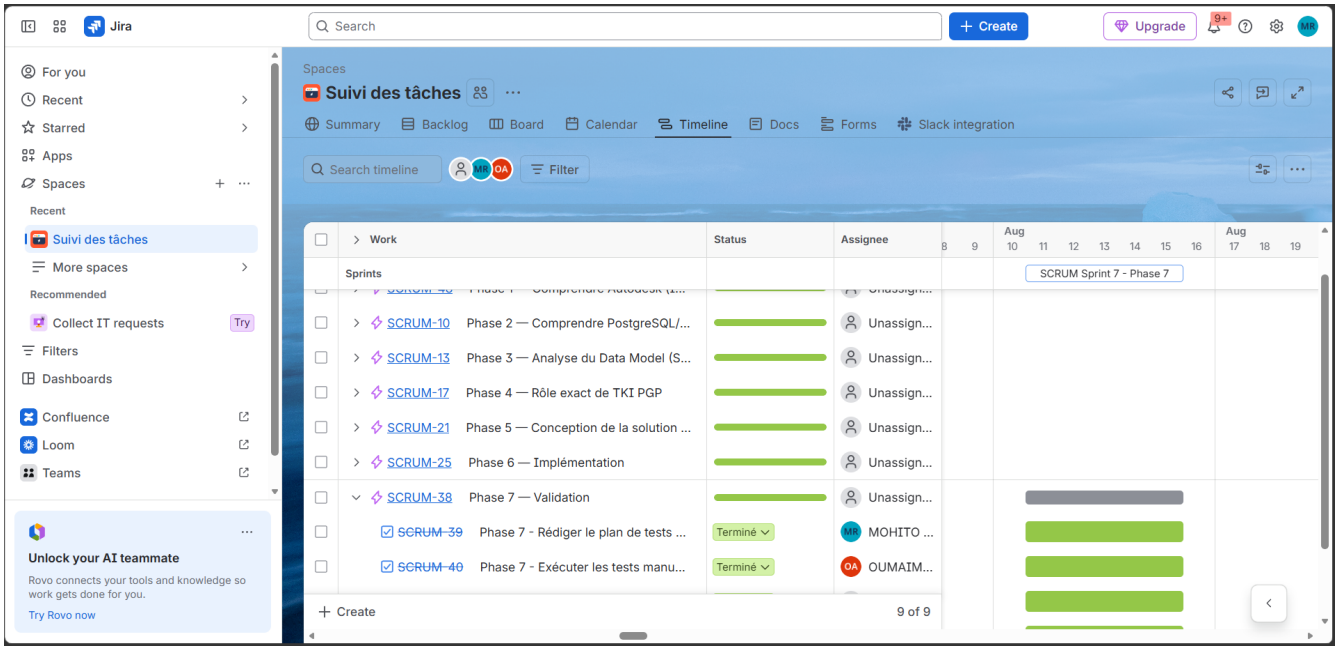


FIGURE 45 – Capture d'écran de Jira

11.3 Planification du projet

Le déroulement du stage, du 1 juillet au 31 août 2026, a été planifié en neuf phases (de la Phase 0 à la Phase 8), suivies dans Jira à l'aide d'épiques et de sprints. Le diagramme de Gantt en présente la chronologie. Les premières phases sont consacrées à la préparation de l'environnement de travail et à la compréhension de l'écosystème Autodesk et de PostgreSQL/PostGIS (Phases 0 à 2), certaines s'exécutant en parallèle, comme les Phases 1 et 2. Elles sont suivies de l'analyse du Data Model par rétro-ingénierie et de l'étude du rôle de TKI PGP (Phases 3 et 4), puis de la conception de la solution alternative (Phase 5), de son implémentation (Phase 6) et de sa validation (Phase 7). La Phase 8 est consacrée à la rédaction du rapport et à la préparation de la soutenance. Chaque phase se termine par un livrable, ce qui a permis de contrôler l'avancement à chaque étape du projet.

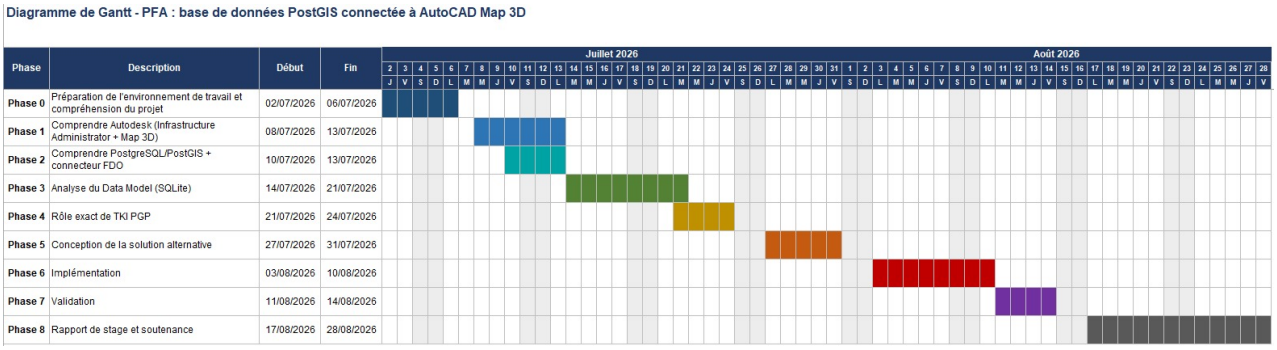


FIGURE 46 – Diagramme de Gantt du projet (juillet – août 2026)

11.4 Communication et collaboration

La communication au sein de l'équipe projet s'est appuyée sur **Slack** pour les échanges quotidiens entre membres du binôme (partage rapide des résultats de tests et résolution des blocages techniques), tandis que les échanges avec les encadrants professionnels se sont déroulés principalement par **courrier électronique** (emails). Des points de synchronisation réguliers ont complété ces échanges informels, permettant de valider les orientations techniques majeures prises tout au long du projet.

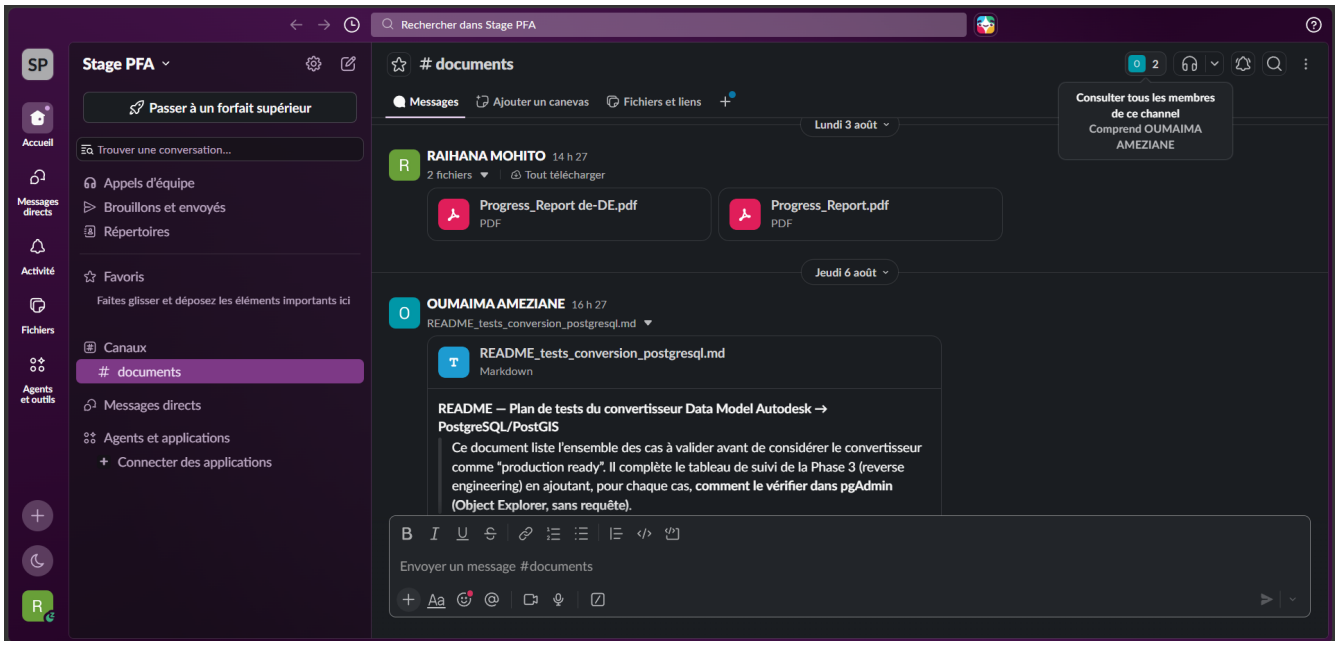


FIGURE 47 – Capture d'écran de Slack

11.5 Gestion des versions et documentation

L'ensemble du code source du projet (scripts de rétro-ingénierie, moteur de conversion `convert_autodesk_to_postgis.py`, service de synchronisation `watch_and_sync.py`, fichiers de configuration Docker) a été versionné à l'aide de **Git**, avec un historique de commits structuré permettant de retracer l'évolution de chaque composant. Cette discipline de versionnage a également facilité la rédaction de la documentation technique du projet, en s'appuyant sur les messages de commit et les jalons identifiés au fil du développement.

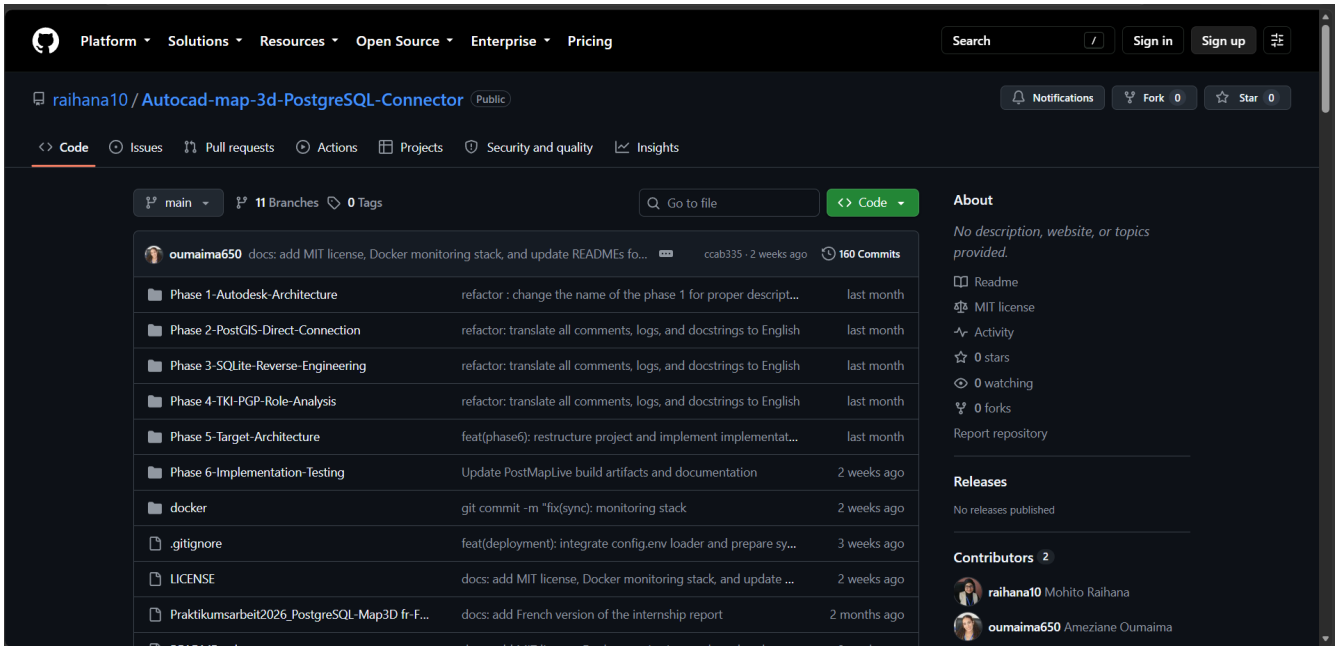


FIGURE 48 – Capture d'écran du dépôt Github

11.6 Synthèse

L'organisation mise en place – découpage itératif du travail, suivi des tâches via Jira, communication continue en presentiel et via Slack , ainsi la gestion rigoureuse des versions via Git – a permis de mener à bien un projet techniquement exigeant dans le temps imparti du stage, tout en conservant une visibilité constante sur l'avancement pour l'ensemble des parties prenantes.

Conclusion générale et perspectives

Ce projet a permis de démontrer, étape par étape, la faisabilité technique d'une alternative open-source au connecteur commercial TKI PGP. La découverte préalable de l'architecture Autodesk et du connecteur FDO natif a permis d'identifier précisément ce qui relève de la simple connectivité spatiale et ce qui relève de la couche métier propre à une Fachschale. La phase de rétro-ingénierie différentielle a ensuite permis d'établir une cartographie fiable et documentée du Data Model Autodesk, condition indispensable à la conception d'un moteur de conversion automatique. Cette compréhension a directement alimenté la conception de l'architecture cible, combinant un générateur de schéma DDL en Python et le connecteur FDO natif pour l'édition temps réel, dont un premier prototype a été validé avec succès sur un Data Model de test. Les travaux de conteneurisation, de supervision et d'organisation projet ont ensuite permis de rapprocher cette solution d'un usage en environnement de production.

Le travail réalisé reste toutefois une **démarche exploratoire**, validée sur un périmètre de test maîtrisé, et non encore une solution industrialisée. Plusieurs pistes restent à approfondir pour la suite du projet : la validation sur un plus large échantillon de Data Models, l'évaluation du comportement de la solution sous des conditions de travail collaboratif réelles (éditions concurrentes, sessions prolongées), la conversion du service d'arrière-plan en véritable service Windows (fonctionnement sans session utilisateur ouverte), la mise en place d'une supervision applicative plus avancée (alerting, tableaux de bord), ainsi qu'une vérification plus fine du périmètre fonctionnel réellement couvert par TKI PGP concernant les règles métier et les formulaires.

Références

1. Infrastructure Administrator Handbuch – Infrastructure Map Server 2016, Autodesk Knowledge Network.
2. Map 3D Toolset in Autodesk AutoCAD – Features, Autodesk.
3. What exactly are these modules used for : Industry Model Setup, Data Editor, Infrastructure Administrator – Autodesk Support.
4. Einführung und erste Schritte – NET Hilfe-Center, TKI Chemnitz.
5. Autodesk Infrastructure Administrator – NET Hilfe-Center, TKI Chemnitz.
6. AutoCAD Map 3D User's Guide – Overview of Features, Autodesk.
7. About Us – Autodesk Company Info, Mission, and Values.
8. TKI Help Center – Introduction PostgreSQL Provider (EN/DE), TKI Chemnitz.
9. TKI Chemnitz – Page produit PGP (EN/DE).
10. TKI Net – Page produit PGP.
11. TKI Help Center – Import/Export de dumps PostgreSQL.
12. TKI Help Center – Section PostgreSQL Provider (PGP).
13. TKI Help Center – General Information about TKI Licensing.